WNDRVR
SUPPORT NETWORK

VXWORKS 7 USB MOUSE CLASS DRIVER TUTORIAL, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 USB Mouse Class Driver Tutorial, SR0650

13 August 2020

1. CREATING THE SKELETON DRIVER CODE

1. About the USB Mouse Class Driver Example

The VxWorks® USB mouse class driver example illustrates how to create a USB class driver using Wind River USB 2.0 API.

The USB mouse class driver included with Wind River USB 2 follows the same steps and concepts introduced in this example, including using helper constructs. For complete information about the Wind River USB API, see the VxWorks-7 API and Configuration Reference.

Prerequisites for using this tutorial are listed below.

- fundamental knowledge of the C language and an understanding of the concept of callbacks, which are used extensively in USB programming
- fundamental knowledge of USB concepts and access to USB standards and documentation available at http://www.usb.org/developers/docs
- experience programming embedded systems
- basic knowledge of VxWorks, including the use of semaphores, the I/O system, and the shell
- access to VxWorks kernel programming documentation, available in the VxWorks dcoumentation
- familiarity with Wind River USB 2.x host configuration

In addition, you will need a build of VxWorks that includes USB host support, and you will need to have started the appropriate host controllers. For details, see Compile the USB Mouse Class Driver and Test It on page 8.

2. Set Up the Environment and Driver Files

The first step in writing a USB mouse class driver is to set up a directory, add files to it, and create a makefile to compile the files.

About This Task

Procedure

1. Create a directory for your driver header and source files.

   By convention, the Wind River USB stack currently puts all USB GEN2 class drivers under *installDir* **/pkgs_v2/connectivity/usb/host/class**, with a subdirectory for each class driver. For the mouse class driver example, create a subdirectory, **mouse**, under **class**. Technically, you can store your driver files in any location that can access the VxWorks header files.

2. Create a makefile.

   Create a makefile under the **mouse** directory and specify the source files to compile.

```
TGT_DIR=installDir/pkgs_v2
DOC_FILES = yourDriverFilename.c
LIB_BASE_NAME = usb2
OBJS = usb2Mse.o
include $(TGT_DIR)/h/make/rules.library
CC_OPTIM                                = $(CC_OPTIM_DRIVER)
```

```
DOC_DIR                                = $(DOCS_ROOT)/vxworks/ref/usb
DOC_CHAPTER                            = USB_libraries
DOC_OPTS                               = $(DOC_OPTS_MAN) -category USB
DOC_AREA_USB                           = usb_drivers
-include $(TGT_DIR)/h/make/config/comps/usb/defs.docs
```

3. Create the driver source file and include the header files.

   Create your class driver source file and add the following required header files to it.

```
#include <vxWorks.h>
#include <selectLib.h>
#include <semLib.h>
#include <logLib.h>
#include <errno.h>
#include <errnoLib.h>
#include <iosLib.h>
#include <rngLib.h>
#include <usb2Mse.h>
#include <usbd.h>
#include <usbOsal.h>
```

   Refer to the VxWorks kernel programming guide documentation to decide which header files are required for your own class driver.

   Note: **usbd.h** already includes the required header file **usbHst.h**. **usb2Mse.h** includes the required header file **usb2Hid.h**, which itself includes the **usb2Helper.h** header file used for the helper routines and structure.

3. About Helper Routines and Structures

When creating the USB mouse class driver, there are several safeguards to consider. Using the helper routine and structure facilitates this task.

Driver Name

The class driver may need to expose a node in the VxWorks I/O system so that user applications can communicate with the driver. For this purpose, the device should have a name associated with it. When more devices of the same class are attached, the name should use a free unit number so that devices can be clearly identified. The class driver should also have a device base name to expose the device name to the VxWorks I/O system.

Protection for Dynamic Attachment and Detachment

Because USB is a dynamically attached and detached system, and devices can come and go asynchronously, it is essential to use a mutex to protect the list of devices while updating that list as devices are added or removed.

Helper Routine

To facilitate writing your driver using a safe method, Wind River provides a helper routine, usb2ClassDriverCreate( ), which creates a special structure, **USB2_CLASS_DRIVER**. The routine takes one argument, which is the default device base name of this class driver.

```
USB2_CLASS_DRIVER * usb2ClassDriverCreate
    (
    char * pDriverDefaultName    /* Default driver iolib base name for driver */
    );
```

This mouse class driver example calls the usb2ClassDriverCreate( ) helper routine in the initialization routine (see Write Code for the Initialization Routine on page 3). The global variable **pUsb2MseDriver** is declared as a pointer to a **USB2_CLASS_DRIVER** structure.

Helper Structure

The **USB2_CLASS_DRIVER** structure is defined in *installDir* **/pkgs_v2/connectivity/usb/host/class/helper/h/usb2Helper.h** as:

```
typedef struct usb2_class_driver
    {
    USBHST_DEVICE_DRIVER hstDriver;     /* Pointer to struct that registers */
                                        /* to the USBD host stack */
    SEM_ID hDriverMutex;                /* Driver Mutex */
    LIST * pDeviceList;                 /* Device list */
    char * pDefaultDriverName;          /* Default driver basename */
    } USB2_CLASS_DRIVER;
```

The first member of this structure is a pointer to the actual class driver structure. This structure, of type **USBHST_DEVICE_DRIVER**, is populated in the initialization routine, documented in Populate the USB Mouse Class Driver Structure on page 5.

The third member of this helper structure is the **pDeviceList**, which is documented in Keep a List of Currently-Connected Devices on page 7.

4. Write Code for the Initialization Routine

After setting up the directory and files for your class driver, you write the initialization routine, which is where you create the driver structure and register the driver.

About This Task

The initialization routine creates the class driver and registers it to the USBD. This routine is called once to register the mouse class driver with the USBD. While multiple instances of the mouse are supported, the driver is initialized only once.

The routines and callbacks in this USB mouse class driver example include debug messages, which allow you to log an error condition or show a warning. Those used in this example are **USB2_MSE_VDBG**, **USB2_MSE_DBG**, **USB2_MSE_WARN**, **USB2_MSE_INFO**, and **USB2_MSE_ERR**. It is desirable to organize these messages into several levels. For information about how to add dynamic debug logging, refer to Debugging with Message APIs.

Finally, this example uses helper structures and routines to facilitate writing the driver. This topic is documented in Debugging with Message APIs.

The completed initialization routine is shown in Complete Initialization Routine.

Procedure

1. Declare a global variable to use for the **USB2_CLASS_DRIVER** structure.

WNDRVR

```
USB2_CLASS_DRIVER * pUsb2MseDriver = NULL;
```

2. Declare a local variable, which is used for the number of the servicing driver.

   This variable is used in the *addDevice* callback (Add the Device to the I/O System) and when installing the driver in the initialization routine (Define the I/O System Methods on page 35).

```
LOCAL int usb2MseDrvNum = 0;
```

3. Add the initialization routine prototype.

   For this example, the initialization routine is named usb2MseInit( ).

```
STATUS usb2MseInit
    (
    char *  pName /* Default base name that will be used, or NULL if no names */
    )
    {
```

4. Declare a local status variable.

```
USBHST_STATUS   hstStatus;
```

5. Check to see if the mouse driver is already initialized.

   For this code, use dynamic debug loggging.

```
    USB2_MSE_VDBG("usb2MseInit(): Initializing the mouse driver...\n",
    1, 2, 3, 4, 5, 6);

    /* If already initialized, ignore and return. */

    if (NULL != pUsb2MseDriver)
        {
        USB2_MSE_WARN("usb2MseInit(): Driver already initialized, return...\n",
                        1, 2, 3, 4, 5, 6);
        return OK;
        }
```

6. Create the driver structure.

   When calling the usb2ClassDriverCreate( ) helper routine to create the driver, first check to see if a valid parameter is given; if it is not, use the driver default name .

```
    if (NULL == pName)
        pUsb2MseDriver = usb2ClassDriverCreate (USB2_MSE_DEVICE_BASE_NAME);
    else
        pUsb2MseDriver = usb2ClassDriverCreate (pName);
```

7. If the driver cannot be created, return an error, otherwise, return **OK**.

```
    if (NULL == pUsb2MseDriver)
        {
        USB2_MSE_ERR("usb2MseInit(): Driver cannot be created\n",
                      1, 2, 3, 4, 5, 6);
        return ERROR;
```


WNDRVR

```
        }

    return OK;
    }
```

Next Steps

When this task is completed, populate the driver structure.

1. Populate the USB Mouse Class Driver Structure on page 5The USB mouse class driver date structure member values are set in the initialization routine.
2. Register the Mouse Class Driver on page 6The USB mouse class driver is registered in the initialization routine.
3. Keep a List of Currently-Connected Devices on page 7A typical USB host class driver must keep track of the currently connected devices.

4.1. Populate the USB Mouse Class Driver Structure

The USB mouse class driver date structure member values are set in the initialization routine.

About This Task

The driver structure, **USBHST_DEVICE_DRIVER**, is documented in *VxWorks 7 USB Host Stack Programmer's Guide:* USB Host Class Driver Registration.

Procedure

1. Set values for the driver members described in *VxWorks 7 USB Host Stack Programmer's Guide:* Setting the Device Type.

   Set the **uProductIDorSubClass** field to **USBD_NOTIFY_ALL**. This macro acts as a wildcard for matching any value in the attached device interface **bInterfaceSubclass** field. The constants for some HID devices are defined in **usb2Hid.h**.

   For this mouse class driver example, the initialization routine sets the following values.

```
    /* Populate the driver structure. */

    pUsb2MseDriver->hstDriver.bFlagVendorSpecific = FALSE;
    pUsb2MseDriver->hstDriver.uVendorIDorClass = USB2_HID_CLASS;
    pUsb2MseDriver->hstDriver.uProductIDorSubClass = USBD_NOTIFY_ALL;
    pUsb2MseDriver->hstDriver.uBCDUSBorProtocol = USB2_MSE_PROTOCOL;
```

2. Assign callbacks to the function pointer data members.

   Do this for the driver add, remove, suspend, and resume callback routines.

```
    pUsb2MseDriver->hstDriver.addDevice = usb2MseDeviceAdd;
    pUsb2MseDriver->hstDriver.removeDevice = usb2MseDeviceRemove;
    pUsb2MseDriver->hstDriver.suspendDevice =  usb2MseSuspendResume;
    pUsb2MseDriver->hstDriver.resumeDevice = usb2MseSuspendResume;
```

   These callbacks still need to be implemented (see Implement Simple Driver Callbacks as Placeholders on page 7).

WNDRVR

Parent topic: Write Code for the Initialization Routine on page 3

Next topic: Register the Mouse Class Driver on page 6

4.2. Register the Mouse Class Driver

The USB mouse class driver is registered in the initialization routine.

About This Task

After filling out the driver structure in the initialization routine, call the usbHstDriverRegister( )to register your class driver.

Procedure

1. Provide a debug statement to indicate that you are registering the driver.

```
/* Register the client with USBD */

    USB2_MSE_VDBG("usb2MseInit(): registering the client with USBD...\n",
                    1, 2, 3, 4, 5, 6);
```

2. Register the driver.

   Call usbHstDriverRegister( ) to register the driver.

```
    hstStatus = usbHstDriverRegister (&pUsb2MseDriver->hstDriver, NULL, "usbGen2Mouse");
```

   - Pass the pointer to the driver structure as the first argument.
   - Pass NULL as the second argument, since this argument is only specified for USB hub class drivers.
   - Pass the pointer to the driver name as the last argument.

   The driver structure and name were created in Write Code for the Initialization Routine on page 3. For additional information, see About Helper Routines and Structures on page 2.

3. Check for success and return the proper value.

```
    if (hstStatus != USBHST_SUCCESS)
        {
        USB2_MSE_ERR("usb2MseInit(): cannot register with USBD...\n",
                    1, 2, 3, 4, 5, 6);

        iosDrvRemove(usb2MseDrvNum, TRUE);
        usb2MseDrvNum = 0;
        usb2ClassDriverDelete(pUsb2MseDriver);

        return ERROR;
        }

    USB2_MSE_DBG("usb2MseInit(): Class driver registration succeeded.\n",
                  1, 2, 3, 4, 5, 6);

    usb2ReleaseDriver(pUsb2MseDriver);
```


WNDRVR

```
    return OK;
    }
```

Parent topic: Write Code for the Initialization Routine on page 3

Previous topic: Populate the USB Mouse Class Driver Structure on page 5
Next topic: Keep a List of Currently-Connected Devices on page 7

4.3. Keep a List of Currently-Connected Devices

A typical USB host class driver must keep track of the currently connected devices.

About This Task

In addition to the interfaces to the USBD, a typical class driver maintains a list of the currently connected devices. This can be done using the **pDeviceList** driver structure member and a linked list facility and the documented in About Helper Routines and Structures. For information about using this list, see Registering Vendor-Specific Devices.

Parent topic: Write Code for the Initialization Routine on page 3

Previous topic: Register the Mouse Class Driver on page 6

5. Implement Simple Driver Callbacks as Placeholders

At this phase in the USB mouse class driver implementation, the driver callbacks are mostly placeholders.

About This Task

There are four callbacks. While the *addDevice* callback requires substantially more code before it is complete, at this point, it is enough to test the driver. Implement the four callback as shown below.

```
/*
 * First pass at implementation of the device add routine.
 */

LOCAL USBHST_STATUS usb2MseDeviceAdd
    (
    UINT32  hDevice,                /* device handler returned by usbd */
    UINT8   bInterfaceNumber,       /* interface number found by the usbd */
    UINT8   uSpeed,                 /* speed of the device */
    void**  ppDriverData            /* place to put the device data */
    )
    {

    USB2_MSE_INFO("Attaching mouse "
                  "(hDevice %p, interface %d, speed %d)\n",
                  hDevice, bInterfaceNumber, uSpeed, 4, 5, 6);

    return USBHST_SUCCESS;
    }
```

```
/*
 * First pass at implmementation of the device remove routine.
 */

void usb2MseDeviceRemove
    (
    UINT32  hDevice,    /* Device handler provided by the USBD */
    void *  pDriverData /* Driver structure returned by add routine */
    )
    {
    USB2_MSE_DBG("Removing Mouse device %p...\n",
                 pDevice, 2, 3, 4, 5, 6);
    return USBHST_SUCCESS;
    }

/*
 * Simple placeholder for suspend and resume.
 */

LOCAL void usb2MseSuspendResume
    (
    UINT32 hDevice,
    void * pDriver)
    {
    USB2_MSE_WARN("usb2MseSuspendResume(): UNSUPPORTED...\n",
                      1, 2, 3, 4, 5, 6);
    }
```

6. Compile the USB Mouse Class Driver and Test It

At this point in the USB mouse class driver example, the basic class driver infrastructure has been established and you can proceed to test the mouse.

Procedure

1. Compile the USB mouse class driver into a VxWorks image.

   Create the VSB and VIP according to the documentation in Adding USB Mouse Device Support. This adds support for many of the helper routines used in this example and which are declared or defined in **usb2Helper.h**.

   You will also want to include the USB SHOW routines in the image. For details, see the section on debugging in the Using the USB SHOW Routines.

2. Boot the VxWorks target.

   When you boot the target, the host controller starts automatically.

3. Connect a USB mouse that conforms to the specification of the USB port.

   Plug the mouse device into a host machine.

   You should see the "Attaching mouse …" message when you insert the device; and you should see the "Removing Mouse …" message when you detach the device. If you see these messages, then you have correctly set up the environment and you proceed writing the driver.

WNDRVR

If you cannot see any messages, check the class codes to ensure that they are set up correctly. Refer to Populate the USB Mouse Class Driver Structure for more information. The remainder of the USB mouse class driver example assumes that you have been able to see these messages and can continue to work with the attached mouse device.

2. UPDATING THE ADDDEVICE CALLBACK

1. Obtain the Device Descriptor Information

In order to complete the USB mouse class driver *addDevice* callback, you first need to obtain information from the device descriptor.

About This Task

Once you have attached the mouse and tested the driver, you can dump and parse the device descriptors. You use this information to properly configure the mouse driver in the *addDevice* callback.

How VxWorks Handles Device Insertion on page 10 Before updating the *addDevice* callback, it is useful to understand how VxWorks handles device insertion and searching for the proper device driver.
Dump the Descriptors on page 11 To update the USB mouse class driver *addDevice* callback you need to dump the descriptors to obtain information required for implementing this callback.
Interpret the Descriptors on page 13 As part of updating the *addDevice* callback, the USB mouse driver reads the device descriptor, each of the configuration descriptors, and each interface in the configuration descriptors.
Compare the Descriptors to the Mouse Interface Definition on page 14 As part of updating the *addDevice* callback for a USB mouse class driver, you must refer to the USB specification for the mouse interface definition.

1.1. How VxWorks Handles Device Insertion

Before updating the *addDevice* callback, it is useful to understand how VxWorks handles device insertion and searching for the proper device driver.

When the USB device is plugged into the host machine, the VxWorks USB stack (the USBD, specifically) performs a number of operations, in the following order.

1. VxWorks detects the insertion of the device.
2. VxWorks resets the device.
3. VxWorks assigns a unique address to the device.
4. VxWorks retrieves the descriptors of the device and learns basic information about it.
5. VxWorks searches for a device driver for the device.

If VxWorks cannot match a driver to a newly inserted device, it simply stops and the device is kept in a list of "orphan devices." VxWorks waits until a device driver is registered for the device before it performs any other action.

If the show commands were added to VIP, then the traces show VxWorks reading the descriptors when the device is first inserted.

The basic driver can issue some commands to the device, but it cannot enable the device like the class driver can.

Even though no class driver matched for the device, there is enough default functionality defined by the USB specification so that descriptors can be dumped and so on. The USB SHOW functionality, documented in Compile the USB Mouse Class Driver and Test It on page 8, takes advantage of this functionality when the descriptors are dumped.

Parent topic: Obtain the Device Descriptor Information on page 10

Related information
Dump the Descriptors on page 11
Interpret the Descriptors on page 13

Compare the Descriptors to the Mouse Interface Definition on page 14

1.2. Dump the Descriptors

To update the USB mouse class driver *addDevice* callback you need to dump the descriptors to obtain information required for implementing this callback.

About This Task

VxWorks stores some of the information available from the configuration of the basic driver, but discards the rest. So you must re-read and parse the descriptors.

The device descriptors provide information about the device and how to program it. Extracting the necessary data from the descriptors and using it to configure the driver is a complex part of USB programming, and is usually done in the *addDevice* callback, as this example demonstrates.

Because each manufacturer lays out their descriptors a bit differently, depending on the features supported by the device, each device may have a different configuration or support for different interfaces and endpoint types. As the class driver programmer, you must take that into consideration when creating the descriptor parsing code.

To view the descriptors, use the usbShow command from Wind River, lsusb from Linux, or UVCView.exe (USBView.exe) from Microsoft. For information about how to use usbShow, refer to the section on debugging, Using the USB SHOW Routines.

Below is an example of the output from usbShow.

```
->usbShow

--------------------------------------------------

  Bus:000 Port:1
  A device is connected to Bus:000 Port:1.
  Node ID: 0x2 (Bus - 0x0 Address - 0x2)
  Descriptor Type: 0x1
  USB Release (BCD): 0x0110
  Device Class: 0x00
  Device SubClass: 0x00
  Device Protocol: 0x00
  Translation of above: Non specific device
  Max Packet Size 0: 0x08 (8)
  Vendor Code: 0x099A
  Product Code: 0x610C
  Device Version: 0x0001
  Manufacturer Index: 0x01 (" ")
  Product Index: 0x02 ("USB Multimedia Keyboard ")
  Serial Number Index: 0x00 ("")
  Number of Configurations: 0x01
  Configuration index 0:
      Config Descriptor Length: 0x09
      Config Descriptor Type: 0x02
      Config Descriptor Total Length: 0x003B (59)
      Config Descriptor Number of Interfaces: 0x02
      Config Descriptor Configuration Value: 0x01
      Config Descriptor String Index: 0x0000 ("")
      Config Descriptor Attribute byte: 0xA0
          Device can be bus powered (valid only for usb1.x)
          Device cannot be self powered
```

```
          Device does have remote wakeup capability
      Config Descriptor Max Power: 0x32 (100 ma)
      --------------------------
      Configuration Interface 0
         Interface Descriptor Length: 0x09
         Interface Descriptor Type: 0x04
         Interface Number: 0x00
         Interface Alternate Setting: 0x00
         Interface numEndpoints: 0x01
         Interface Class: 0x03
         Interface SubClass: 0x01
         Interface Protocol: 0x01
         Translation of above: Human Interface device
         Interface Index: 0x00 ("")
         Endpoint Index: 0
           Endpoint Descriptor length: 0x07
           Endpoint Descriptor Type: 0x05
           Endpoint Address Mask: 0x81
               Endpoint number: 0x01
               Endpoint Direction: IN
           Attributes Mask: 0x03
               Transfer Type: Interrupt.
               Synchronization Type: No Synchronization.
               Usage Type: Data Endpoint.
           Maximum Packet Size: 0x0008 (8)
           Interval: 0x0A
      --------------------------
      Configuration Interface 1
         Interface Descriptor Length: 0x09
         Interface Descriptor Type: 0x04
         Interface Number: 0x01
         Interface Alternate Setting: 0x00
         Interface numEndpoints: 0x01
         Interface Class: 0x03
         Interface SubClass: 0x01
         Interface Protocol: 0x02
         Translation of above: Human Interface device
         Interface Index: 0x00 ("")
         Endpoint Index: 0
           Endpoint Descriptor length: 0x07
           Endpoint Descriptor Type: 0x05
           Endpoint Address Mask: 0x82
               Endpoint number: 0x02
               Endpoint Direction: IN
           Attributes Mask: 0x03
               Transfer Type: Interrupt.
               Synchronization Type: No Synchronization.
               Usage Type: Data Endpoint.
           Maximum Packet Size: 0x0005 (5)
           Interval: 0x0A
--------------------------------------------------
```

However, because the output is large, the output is shown again below, but with only the descriptors of interest:

```
-------------------------------------------------

  . . .
  Device Class: 0x00
      . . .
      --------------------------
      Configuration Interface 0
         . . .
         Interface Class: 0x03
         Interface SubClass: 0x01
         Interface Protocol: 0x01
         . . .
      --------------------------
      Configuration Interface 1
         . . .
         Interface Class: 0x03
         Interface SubClass: 0x01
         Interface Protocol: 0x02
         Translation of above: Human Interface device
               . . .
               Endpoint Direction: IN
           . . .
-------------------------------------------------
```

Parent topic: Obtain the Device Descriptor Information on page 10

Related information

How VxWorks Handles Device Insertion on page 10
Interpret the Descriptors on page 13
Compare the Descriptors to the Mouse Interface Definition on page 14

1.3. Interpret the Descriptors

As part of updating the *addDevice* callback, the USB mouse driver reads the device descriptor, each of the configuration descriptors, and each interface in the configuration descriptors.

About This Task

This mouse class driver example uses the simplest configuration:

- Class is **HID**
- Subclass is **Boot**
- Protocol is **Mouse**

The driver only configures the **Interrupt IN** endpoint.

The "Device Class" descriptor is 0:

```
Device Class: 0x00
```

which means that the device class is present in the interface descriptor of the selected configuration (normally the current configuration, as most devices use only one configuration). As this is a composite keyboard with an integrated mouse, we can see two interfaces in the descriptor dump.

Procedure

1. Ignore the first interface, which is a keyboard.

   The first interface has an "Interface Class" of 3, an "Interface Subclass" of 1, and an "Interface Protocol" of 1. These values are defined in the documents at USB.ORG. The HID (Human Interface Device) has a class value of 3. The subclass is a "boot protocol" HID device and the "Interface Protocol" of 1 indicates that it is a boot protocol keyboard.

2. Use the second interface, which is the mouse.

   The second interface has an "Interface Class" of 3, an "Interface Subclass" of 1, and an "Interface Protocol" of 2, which indicates it is a mouse. So this is used for the current example. The steps in section Obtain the Interface Descriptor on page 22 show how to find the desired interface descriptor programmatically.

Parent topic: Obtain the Device Descriptor Information on page 10

Related information

How VxWorks Handles Device Insertion on page 10
Dump the Descriptors on page 11
Compare the Descriptors to the Mouse Interface Definition on page 14

1.4. Compare the Descriptors to the Mouse Interface Definition

As part of updating the *addDevice* callback for a USB mouse class driver, you must refer to the USB specification for the mouse interface definition.

About This Task

The figure below shows the mouse interface definition from the USB HID 1.11 class specification. The descriptors fully define the information needed to program the I/O for the mouse.

E.7 Interface Descriptor (Mouse)

Part	Offset/Size (Bytes)	Description	Sample Value
bLength	0/1	Size of this descriptor in bytes.	0x09
bDescriptorType	1/1	Interface descriptor type (assigned by USB).	0x04
bInterfaceNumber	2/1	Number of interface.	0x01
bAlternateSetting	3/1	Value used to select alternate setting.	0x00
bNumEndpoints	4/1	Number of endpoints.	0x01
bInterfaceClass	5/1	Class code (HID code assigned by USB).	0x03
bInterfaceSubClass	6/1	1 = Boot Interface subclass.	0x01
bInterfaceProtocol	7/1	2 = Mouse.	0x02
iInterface	8/1	Index of string descriptor.	0x00

Specifically, you can see the following elements:

bNumEndpoints

Number of endpoints, value 0x01.

bInterfaceClass

Class code (HID code assigned by USB), value 0x03.

bInterfaceSubClass

1 = Boot Interface subclass, value 0x01.

bInterfaceProtocol

2 = Mouse, value 0x02.

The interface descriptor has one endpoint, which is an IN (into the host machine). This is an interrupt endpoint, which is polled at 10 ms intervals with a maximum buffer size of 5.

Note: The boot protocol mentioned above is defined in the Device Class Definitions of Human Interface Devices (HID) version 1.11 available from USB.ORG at: http://www.usb.org/developers/hidpage/.

Parent topic: Obtain the Device Descriptor Information on page 10

Related information

How VxWorks Handles Device Insertion on page 10
Dump the Descriptors on page 11
Interpret the Descriptors on page 13

2. Set the Configuration and Interface

After obtaining the device descriptor information for the USB mouse class driver, you are ready to continue to implement the *addDevice* callback.

About This Task

For each USB mouse connected to the system, the USBD attaches each interface of the device to the appropriate class driver *addDevice* callback. The device is then mounted in the **iosLib** and the application can start performing I/O requests to the device driver.

The *addDevice* callback is, therefore, responsible for identifying the device, configuring it, and creating the device structure that enables use of the device; and this callback routine is where the descriptor parsing is normally handled. So, after creating the basic implementation (Implement Simple Driver Callbacks as Placeholders on page 7), you continue to update the *addDevice* callback to configure the driver for the mouse.

1. Examine the Device Information Structure on page 16The *addDevice* callback is where you configure the USB mouse class driver using information from the device information structure. Thus, you should understand the relevant members of this structure.
2. Retrieve the Device Information Structure on page 17After reviewing the device information structure, you can update the *addDevice* callback to retrieve this structure for the USB mouse class driver.
3. Set the Configuration on page 18After retrieving the device information structure, continue updating the implementation of the *addDevice* callback to set the current configuration for the USB mouse class driver.
4. Set the Interface on page 19After setting the configuration in the USB mouse class driver *addDevice* callback, you set the interface.

WNDRVR

2.1. Examine the Device Information Structure

The *addDevice* callback is where you configure the USB mouse class driver using information from the device information structure. Thus, you should understand the relevant members of this structure.

Procedure

1. Examine the **USBD_DEVICE_INFO** structure.

   The **USBD_DEVICE_INFO** is generated by USBD to represent the device associated with **hDevice**. This structure is defined in the *installDir* **/pkgs_v2/connectivity/usb/host/core/h/usbd.h** header file, which was included in your class driver (see Set Up the Environment and Driver Files on page 1).

   Because this structure is used by both the class driver and the USBD, some members do not pertain to the class driver. Those are omitted below, and only the elements related to the descriptor parsing are shown.

   Note: The information in this structure is intended to be READ-ONLY by class drivers, so do not modify any elements directly. If you do, USBD functionality may be affected and results are undefined.

```
typedef struct usbd_device_info
{
…omitted…

UINT16  uBCDUSB;           /* bcdUSB, USB version device conforms to */
UINT8   uMaxPacketSize0;   /* Max packet size for default control endpoint */
UINT8   uDeviceClass;      /* Class code of the device */
UINT8   uDeviceSubClass;   /* Sub class code of the device */
UINT8   uDeviceProtocol;   /*   Protocol code of the device */
UINT16  uVendorID;         /* Vendor Id of the device */
UINT16  uDeviceID;         /* Device Id for device */
UINT16  uBCDDevice;        /* USB device release version binary coded decimal */

…omitted…

UINT8   uInterfaceCount;                /* # of interfaces supported */
UCHAR   pBOS;                           /* Ptr to Binary Device Object Store */
UCHAR * pCurrentConfiguration;          /* Ptr to hold config of the device */
UINT8   uNumConfigurations;             /* # of possible configurations */
pUSBD_INTERFACE_INFO pInterfacesInfo;   /* Info about interfaces */

        …omitted…

        } USBD_DEVICE_INFO, *pUSBD_DEVICE_INFO;
```

2. Review the first eight members.

   Except **uMaxPacketSize0**, the values for the initial eight members of this structure are retrieved from the device descriptors. These are useful if you are writing a vendor-specific device driver or a standard class driver, for which the device shows its class information in the device descriptor. Since most devices report class information in the interface descriptor level, the variables are clearly named.

3. Review the configuration members.

   The second half of the device information structure contains several members used when configuring the device driver. These members are described below.

uNumConfigurations

Indicates how many configurations the device supports, since a USB device can support multiple configurations. However, most devices use a single configuration and **uNumConfigurations** is typically set to 1.

pCurrentConfiguration

Specifies a pointer to a data buffer for the whole configuration data of the current configuration, including all the interface descriptors and their endpoint descriptors, as well as any class or vendor-specific descriptors for this configuration. For more information, see Set the Configuration on page 18.

pInterfacesInfo

Specifies an array of structures of type **USBD_INTERFACE_INFO**, which is dynamically allocated by the USBD with the actual number of interfaces in the current configuration of the device. For more information, see Set the Interface on page 19.

4. Note that, on attachment, the device is in its default configuration.

   Regardless of the number of configurations a device supports, there can be only one configuration active at a time. When the device is attached, it should be at its default configuration. So when the *addDevice* callback is invoked, it is still in its default configuration.

Parent topic: Set the Configuration and Interface on page 15

Next topic: Retrieve the Device Information Structure on page 17

2.2. Retrieve the Device Information Structure

After reviewing the device information structure, you can update the *addDevice* callback to retrieve this structure for the USB mouse class driver.

About This Task

Update the implementation of the *addDevice* callback that you wrote in Implement Simple Driver Callbacks as Placeholders on page 7 to add code to retrieve the **USBD_DEVICE_INFO** structure.

Procedure

1. Declare a device information variable.

   After the prototype, declare a variable, **pDeviceInfo** of type **pUSBD_DEVICE_INFO** and initialize it to **NULL**.

```
    pUSBD_DEVICE_INFO  pDeviceInfo = NULL;
```

2. Call usbdTranslateDeviceHandle( ) to retrieve the device information structure.

   Add this call after the information message about attaching the mouse.

```
/* Get the device information, for the descriptors */

    usbdTranslateDeviceHandle(hDevice, &pDeviceInfo);
```

3. Add debugging code to ensure that the mouse is still connected.

WNDRVR

```
if (NULL == pDeviceInfo )
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): Mouse is apparently "
             "not connected anymore, the USBD pDeviceInfo is NULL\n",
             1, 2, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }
```

Parent topic: Set the Configuration and Interface on page 15

Previous topic: Examine the Device Information Structure on page 16
Next topic: Set the Configuration on page 18

2.3. Set the Configuration

After retrieving the device information structure, continue updating the implementation of the *addDevice* callback to set the current configuration for the USB mouse class driver.

About This Task

This mouse example has only one configuration. Nevertheless, according to the USB specification, this configuration must still be selected, even though it is already the current configuration.

Procedure

1. Declare current configuration and index variables.

   Declare a **pCurrentConfiguration** variable and a **configurationIndex** variable, and initialize them to **NULL** and **0**.

```
    pUSBHST_CONFIG_DESCRIPTOR        pCurrentConfiguration = NULL;
    UINT8        configurationIndex = 0;
```

2. Retrieve the current configuration.

   Use **pCurrentConfiguration** to retrieve the current configuration member from the **pDeviceInfo** structure, described in Examine the Device Information Structure on page 16.

   Because the **pCurrentConfiguration** member of the **USBD_DEVICE_INFO** is declared as a **UCHAR ***, this example first casts it to a **pUSBHST_CONFIG_DESCRIPTOR** before assigning its value to the local **pCurrentConfiguration** variable.

```
pCurrentConfiguration = (pUSBHST_CONFIG_DESCRIPTOR)
                                pDeviceInfo->pCurrentConfiguration;
```

   For details on **pUSBHST_CONFIG_DESCRIPTOR**, see Standard Configuration Descriptor Structure on page 58.

3. Add debugging code to ensure that the configuration was properly retrieved.

```
if (NULL == pCurrentConfiguration)
    {
    USB2_MSE_ERR("usb2MseDeviceAdd(): pCurrentConfiguration NULL\n",
                     1, 2, 3, 4, 5, 6);
```


WNDRVR

```
    return USBHST_FAILURE;
    }
```

4. Form the correct index number for the desired configuration.

   The second parameter that you will need to set the index for the desired configuration is the actual index. To form this index number, subtract 1 from the **bConfigurationValue** of the desired configuration.

```
    /* Form the correct index number. */

    configurationIndex = pCurrentConfiguration->bConfigurationValue - 1;
```

   Note that **bConfigurationValue** is a member of the structure, **pUSBHST_CONFIG_DESCRIPTOR** (see Standard Configuration Descriptor Structure on page 58). So, use **pCurrentConfiguration** to access its **bConfigurationValue** member.

5. Set the desired configuration.

   Now that you have the correct parameter, call usbHstSetConfiguration( ) to set the configuration.

   Note that the first argument, **hDevice**, is the handle to the device used to communicate with the USBD. This was passed as the first argument to the *addDevice* callback. The second argument to usbHstSetConfiguration( ) is the **configurationIndex** variable.

```
    /* Select the desired configuration. */

if (usbHstSetConfiguration (hDevice,configurationIndex ) != OK)
    {
    USB2_MSE_ERR("usb2MseDeviceAdd(): Failed to set configuration - "
                 "device not added.\n", 1, 2, 3, 4, 5, 6);

    return USBHST_FAILURE;
    }
```

   The **usbHstSetConfiguration** routine is used to issue a **SetConfiguration** USB standard request, as described in *VxWorks 7 USB Host Stack Programmer's Guide:* Using Data Transfer Interfaces.

Parent topic: Set the Configuration and Interface on page 15

Previous topic: Retrieve the Device Information Structure on page 17
Next topic: Set the Interface on page 19

2.4. Set the Interface

After setting the configuration in the USB mouse class driver *addDevice* callback, you set the interface.

About This Task

For the USB mouse class driver, you can assume that the device has only one interface for the mouse functionality, with only one default setting.

Procedure

1. Declare a variable to keep track of the status when validating calls.

WNDRVR

```
USBHST_STATUS                       hstStatus = USBHST_FAILURE;
```

2. Declare a **pInterface** variable as a pointer to a **USBD_INTERFACE_INFO** structure.

   Declare this variable and set it to **NULL**, as follows:

```
    pUSBD_INTERFACE_INFO            pInterface = NULL;
```

   The **USBD_INTERFACE_INFO** structure used for the mouse is shown below.

```
typedef struct usbd_interface_info
        {
    UINT8      uInterfaceNum;       /* Interface number */
    UINT8      uAlternateSetting;   /* Alt settting value for this interface */
    UINT8      uNumEndpoints;       /* # of end points in the interface */
    UINT8      uInterfaceClass;     /* Class code for the interface */
    UINT8      uInterfaceSubClass;  /* Sub class ocdecode for the interface */
    UINT8      uInterfaceProtocol;  /* protocol code  for the interface */

        …omitted…

        } USBD_INTERFACE_INFO, *pUSBD_INTERFACE_INFO;
```

3. Recall that **pInterfacesInfo** is a member of the **USBD_DEVICE_INFO** structure.

   Refer to Examine the Device Information Structure. You see that **pInterfacesInfo** is an array of structures of type **USBD_INTERFACE_INFO**.

4. Recall that **bInterfaceNumber** is passed to the *addDevice* callback.

   Refer to Complete addDevice Callback. You see that **bInterfaceNumber** identifies the interface by number, to distinguish between different interfaces in the same configuration.

5. Use **bInterfaceNumber** to obtain the necessary information

   Pass **bInterfaceNumber** as the index number into **pInterfacesInfo**, to obtain a pointer to the correct **USBD_INTERFACE_INFO** structure for this interface. Then, assign that to the new **pInterface** variable, as shown in the code below.

```
pInterface = &(pDeviceInfo->pInterfacesInfo[bInterfaceNumber]);
```

6. Add debugging code to ensure **pInterface** is valid.

```
    if (NULL == pInterface)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd():No Interface %d\n",
                     bInterfaceNumber, 2, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }
```

7. Declare and initialize a **uAlternateSetting** variable.

```
    UINT8            uAlternateSetting = 0;
```

8. Set the value for this alternate setting variable.

   Now that **pInterface** points to the correct structure, use it to set the alternate setting.

WNDRVR

```
    uAlternateSetting = pInterface->uAlternateSetting;
```

9. Set the interface.

   Now that you have the correct parameter, call usbHstSetInterface( ) to set the interface. Include debugging code as shown in the code snippet below.

   The first two arguments to usbHstSetInterface( ) are the first two arguments that were passed to the *addDevice* callback. The third argument to usbHstSetInterface( ) is the **uAlternateSetting** variable.

```
    /* Following is the code that does the SetInterface standard USB request */

    USB2_MSE_VDBG("usb2MseDeviceAdd(): Setting Interface %d \n",
                    bInterfaceNumber, 2, 3, 4, 5, 6);

    /* Set the desired interface alternate setting */

    if  ((hstStatus = usbHstSetInterface(hDevice,
                                  bInterfaceNumber,
                                  uAlternateSetting))
        != USBHST_SUCCESS)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): Failed to set interface - "
                    "device not added \n", 1, 2, 3, 4, 5, 6);
```

   The usbHstSetInterface( ) routine is used to issue a SetInterface( ) USB standard request, as described in Using Standard Request Interfaces.

10. Check for a STALL.

    If a device only supports a default setting for the specified interface, then a STALL may be returned in the status stage of the request. In this case, you can simply continue, since this is a control pipe protocol STALL and is automatically cleared when the device receives the next setup packet.

```
        if (hstStatus != USBHST_STALL_ERROR)
            return USBHST_FAILURE;
        }
```

Parent topic: Set the Configuration and Interface on page 15

Previous topic: Set the Configuration on page 18

3. Obtain the Endpoint Address

The next step in implementing the *addDevice* callback for the USB mouse class driver is to find the endpoint address.

About This Task

Once the interface is set, the device is ready work with the desired interface, along with a set of endpoints defined by the class document. You next need to find the endpoint address, which is done in the *addDevice* callback.

Class driver developers first should refer to the class document to find out how many interfaces and what type of endpoints are used for your particular device class. This mouse example uses one interface for the mouse function and one Interrupt IN endpoint to report the mouse data.

The class document only defines the number and type of endpoints used in the interface, it does not define the actual endpoint address. To communicate with the device, the class driver needs to find the actual endpoint address. To do this, you first need to obtain the interface descriptor.

Obtain the Interface Descriptor on page 22 The first step in obtaining the endpoint address for the *addDevice* callback is to obtain the interface descriptor.
Parse the Class or Vendor Specification Descriptors on page 23 After obtaining the interface descriptor, the next step in implementing the USB mouse class driver *addDevice* callback is to determine if your class has a definition for any class- or vendor-specific descriptors.
Parse the Configuration Descriptor on page 24 After handling class- or vendor-specific descriptors, the next step in implementing the USB mouse class driver *addDevice* callback is to parse the configuration descriptor, primarily to find the usable endpoints with the desired interface.
How the Endpoint Helper Routine Works on page 25 The helper routine, usb2FindEndpoint( ), is used to find the endpoint when parsing the configuration descriptors for the USB mouse class driver.

3.1. Obtain the Interface Descriptor

The first step in obtaining the endpoint address for the *addDevice* callback is to obtain the interface descriptor.

Procedure

1. Use the current configuration pointer.

   Recall that the entire configuration buffer for the selected configuration value was already been retrieved in the *addDevice* callback; and it can be accessed using the **pCurrentConfiguration** pointer (see Set the Configuration on page 18).

   The USB specification defines the configuration, interface and endpoint descriptors in a well organized manner in the configuration buffer, so that it is possible to iterate through the buffer to find the desired descriptor.

2. Declare a pointer to the interface descriptor.

   In the *addDevice* callback, declare this variable as follows:

```
USBHST_INTERFACE_DESCRIPTOR *       pInterfaceDesc;
```

3. Call a helper routine to find the interface descriptor.

   To locate the interface descriptor of the desired interface alternate setting, you can use a helper routine, **usb2FindInterface**, provided in the GEN2 class driver helper module and declared in *installDir* **/pkgs_v2/connectivity/usb/host/class/helper/h/usb2Helper.h**.

   The following code shows you how to use it.

```
    USB2_MSE_VDBG("usb2MseDeviceAdd(): Searching for endpoints... \n",
                  1, 2, 3, 4, 5, 6);

    pInterfaceDesc = usb2FindInterface(pDeviceInfo,
                                       bInterfaceNumber,
                                       uAlternateSetting);
```

   The arguments to usb2FindInterface( ) should be familiar, since they have all been previously used:

- **pDeviceInfo** is a pointer to the information associated with this device, **hDevice** (see Retrieve the Device Information Structure on page 17)
- **bInterfaceNumber** is the interface number (see Set the Interface on page 19)
- **uAlternateSetting** is the alternate setting, which is inside the configuration buffer (see Set the Interface on page 19)

4. Add debugging code to validate the interface descriptor return value.

   The **pInterfaceDesc** returned by this routine is a pointer to the desired interface descriptor. Add debugging code to validate this return value.

```
if (NULL == pInterfaceDesc)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): No such interface"
                     " bInterfaceNumber %d, uAlternateSetting %d on device\n",
                     (int) bInterfaceNumber, uAlternateSetting, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }
```

Parent topic: Obtain the Endpoint Address on page 21

Related information

Parse the Class or Vendor Specification Descriptors on page 23
Parse the Configuration Descriptor on page 24
How the Endpoint Helper Routine Works on page 25

3.2. Parse the Class or Vendor Specification Descriptors

After obtaining the interface descriptor, the next step in implementing the USB mouse class driver *addDevice* callback is to determine if your class has a definition for any class- or vendor-specific descriptors.

About This Task

The class- or vendor specification may define some class- or vendor specific descriptors immediately following the interface descriptor. For example, the HID class specification defines an HID descriptor following the interface descriptor.

- If there are no class- or vendor-specific descriptors following the interface descriptor, the endpoint descriptors for that interface immediately follow the interface descriptor.
- If there are any class- or vendor-specific descriptors, then the endpoint descriptor appears immediately after the end of the last class-specific (or vendor-specific) descriptor.

For this mouse example this step is not necessary, and you can move on to Parse the Configuration Descriptor on page 24. However, if you are writing your own class driver, and your class has a definition for any class- or vendor specific descriptors, some intermediate steps are required.

Procedure

1. Parse the class-specific descriptors.

   Parse the class-specific (or vendor-specific) descriptors from **pInterfaceDesc**, adding an offset of sizeof(USBHST_INTERFACE_DESCRIPTOR) to reach the start of the class specific descriptors. Following is the example code for this step:

WNDRVR

```
char * pClassSpecificDescrs = ((char *)pInterfaceDesc) + sizeof(USBHST_INTERFACE_DESCRIPTOR);
```

2. Convert the buffer to the correct type.

   Next, convert the buffer to the pointer type of your class- or vendor-specific descriptors.

```
USBHST_DESCRIPTOR_HEADER * pDescrHeader = (USBHST_DESCRIPTOR_HEADER *) pClassSpecificDescrs;
```

3. Handle multiple class- or vendor-specific descriptors.

   Note that there maybe several such class- or vendor specific descriptors. In this case, you need to use the **bLength** and **bDescriptorType** fields to validate whether it is the one you want to use. If you do not want to sue it, move **bLength** further to the next descriptor, and then do the same check.

Parent topic: Obtain the Endpoint Address on page 21

Related information

Obtain the Interface Descriptor on page 22
Parse the Configuration Descriptor on page 24
How the Endpoint Helper Routine Works on page 25

3.3. Parse the Configuration Descriptor

After handling class- or vendor-specific descriptors, the next step in implementing the USB mouse class driver *addDevice* callback is to parse the configuration descriptor, primarily to find the usable endpoints with the desired interface.

About This Task

For the mouse class driver example, the HID 1.11 specification gives an example of possible mouse descriptors. The mouse has an HID descriptor that immediately follows the interface descriptor. However, this example mouse class driver only uses the Boot Protocol of the mouse. In this case, you can ignore the HID descriptor. and go on to locate the endpoints defined in the class specification.

Recall that the mouse had an endpoint type of Interrupt IN. So, the following steps update the *addDevice* callback to find this interrupt endpoint (and some additional attributes).

Procedure

1. Declare a pointer to an endpoint descriptor.

   In the *addDevice* callback, declare this variable as follows:

```
USBHST_ENDPOINT_DESCRIPTOR *        pEndPointDesc;
```

   The **USBHST_ENDPOINT_DESCRIPTOR** structure is defined in *installDir* **/pkgs_v2/connectivity/usb/core/h/usbHst.h** and contains a data member that is the endpoint address on the USB device.

2. Call the usb2FindEndpoint( ) helper routine.

   So, add the following code to the *addDevice* callback.

WNDRVR

```
/* Find the must-have Interrupt IN endpoint */

pEndPointDesc = usb2FindEndpoint((UCHAR* )pInterfaceDesc,
                                        USB2_ATTR_INTERRUPT,
                                        USB2_ENDPOINT_IN,
                                        0,
                                        0,
                                        0);
```

For details on this helper routine, see How the Endpoint Helper Routine Works on page 25.

3. Add debugging code to validate the **pEndPointDesc**.

```
if (NULL == pEndPointDesc)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): NO IN endpoint found\n",
                     1, 2, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }
```

4. Declare and initialize a variable to use for the endpoint address.

   In the *addDevice* callback, declare and initialize the following variable, which will be used to hold the endpoint's address.

```
    UINT8                               uInEndPointAddr = 0;
```

5. Assign the endpoint address to the **uInEndPointAddr** variable.

   Once you have retrieved the endpoint descriptor structure, you can now access the endpoint address data member, **bEndpointAddress**, and assign it to the **uInEndPointAddr** variable.

```
uInEndPointAddr = pEndPointDesc->bEndpointAddress;
```

Parent topic: Obtain the Endpoint Address on page 21

Related information

Obtain the Interface Descriptor on page 22
Parse the Class or Vendor Specification Descriptors on page 23
How the Endpoint Helper Routine Works on page 25

3.4. How the Endpoint Helper Routine Works

The helper routine, usb2FindEndpoint( ), is used to find the endpoint when parsing the configuration descriptors for the USB mouse class driver.

This helper routine is used in the task documented in Parse the Configuration Descriptor on page 24. This routine is declared as follows:

```
USBHST_ENDPOINT_DESCRIPTOR * usb2FindEndpoint
    (
    UINT8 * pInterfaceStart,            /* Ptr to interface buffer in config */
    UINT8   uTransferType,             /* Transfer type           */
    UINT8   uTransferDir,                /* Transfer direction      */
    UINT8   uSyncType,                   /* Synchronisation Type (Iso Mode) */
```


WNDRVR

```
    UINT8   uUsageType,                 /* Usage Type (Iso Mode)   */
    UINT8   uMatchEndpointIndex  /* index of matching endpoint type */
    );
```

The attributes are drawn from the USB standard endpoint descriptor fields.

For example, we can specify the **uTransferType** parameter as **USB2_ATTR_INTERRUPT** because we are finding an interrupt endpoint. The **uTransferDir** can simply takes **USB2_ENDPOINT_OUT** or **USB2_ENDPOINT_IN**. The **uSyncType** and **uUsageType** are for isochronous endpoints where further attributes like Synchronization and Usage types are required; for non-isochronous endpoints these parameters are not used.

Normally, a class does not define more than one endpoint of the same transfer type and transfer direction in the same interface (for example, two bulk in endpoints in the same interface). However, this is not limited by the specification, so it must be taken into account. sIn a case like this, the **uMatchEndpointIndex** can be used to specify the one you are searching for. Typically, there is one unique endpoint in the interface, so that you can set **uMatchEndpointIndex** to 0—that is to find the first endpoint that matches the other endpoint attributes set aside.

This example also uses the helper routine usb2FindEndpointHelperDescr( ) to find a helper descriptor for a given endpoint. And example of this is the UAS Pipe Usage Descriptor for a UAS endpoint.

The macros that are to be used along with **usb2FindEndpoint** are defined in *installDir* **/pkgs_v2/connectivity/usb/host/class/helper/h/usb2Helper.h**.

Parent topic: Obtain the Endpoint Address on page 21

Related information

Obtain the Interface Descriptor on page 22
Parse the Class or Vendor Specification Descriptors on page 23
Parse the Configuration Descriptor on page 24

4. Set up a Pipe for Initiating Transfers

After parsing the configuration descriptor to the endpoint, you continue implementing the USB mouse class driver *addDevice* callback by setting up a pipe for the endpoint.

Procedure

1. Declare a packet size variable.

   The endpoint descriptor structure, **USBHST_ENDPOINT_DESCRIPTOR** (see Parse the Configuration Descriptor on page 24), has a data member, **wMaxPacketSize**, which is a UINT16 that holds the endpoint's maximum packet size.

   Declare a s variable to use to obtain and convert this value to the correct type.

   ```
   UINT16                              wMaxPacketSize;
   ```

2. Convert the maximum packet size to the correct type.

   The following code (macro) converts a 16-bit integer from little endian format to the CPU format. Pass the endpoint's maximum packet size data member as the argument.

   ```
   wMaxPacketSize = OS_UINT16_LE_TO_CPU(pEndPointDesc->wMaxPacketSize);
   ```

The **OS_UINT16_LE_TO_CPU** macro is defined in *installDir* **/vxworks-7/pkgs_v2/connectivity/usb/core/h/usbOsal.h**.

3. Declare a variable for the transfer structure.

```
USB_TRANSFER_SETUP_INFO             xferSetup;
```

The **USB_TRANSFER_SETUP_INFO** structure is defined in *installDir* **/pkgs_v2/connectivity/usb/core/h/usbHst.h** and contains three data members.

4. Initialize the memory for the transfer structure.

The following code (macro) initializes the memory with specified value. It takes a pointer to the source memory location, a value to be assigned to the memory location, and the number of bytes to be copied.

```
    OS_MEMSET(&xferSetup, 0, sizeof(USB_TRANSFER_SETUP_INFO));
```

The **OS_MEMSET** macro is defined in *installDir* **/vxworks-7/pkgs_v2/connectivity/usb/core/h/usbOsal.h**.

5. Populate the transfer structure.

The transfer structure has three members, a maximum possible transfer size for the pipe, a maximum possible transfer requests at a time, and any required flags.

```
    xferSetup.uMaxTransferSize = max(_CACHE_ALIGN_SIZE, wMaxPacketSize);
    xferSetup.uMaxNumReqests = 1;
    xferSetup.uFlags = 0;
```

6. Set up the data transfer requirement for the endpoint.

Call usbHstPipePrepare( ) to set up the data transfer requirements. This routine is defined in *installDir* **/pkgs_v2/connectivity/usb/core/h/usbHst.h**. For more information, see the entry in the online API reference.

```
    hstStatus = usbHstPipePrepare(hDevice,
                    pEndPointDesc->bEndpointAddress,
                    &xferSetup);
```

7. Add debugging code to validate the call.

```
    if (hstStatus != USBHST_SUCCESS)
        {
        USB2_MSE_ERR("Interrupt IN at 0x%x set up failed\n",
            pEndPointDesc->bEndpointAddress, 2, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }
```

5. Decide How to Implement Input and Output

After parsing the device descriptors, and finding the desired interface and endpoint information that matches the class specification definitions, the next step in implementing the *addDevice* callback is to evaluate the best approach for handling I/O.

About This Task

Up to now, all communication —such as retrieving descriptors, setting configuration, setting the interface, and so on—has been done using the default control pipe. No actual class defined I/O requests have yet been performed. So the next step is to decide how to implement this communication, for which you should consider how to support the functional characteristics of the device and how to best implement the driver to integrate into VxWorks.

Evaluate Your Device Types

You first need to evaluate the functional requirements of the mouse driver. For this, you must select the right driver model for the device. You should first evaluate the functional requirements of your device. The functional requirements of the mouse are listed below and must be considered when writing the driver:

- The mouse driver shall support asynchronous attach/removal from the USB.

The mouse driver shall support standard POSIX open( ) to the attached device, and read( ) for the mouse data. In addition it should support close( ), which closes the FD to the device, cancels all active URBs, and releases the device resources. The write( ) is not supported, and will return with error when called.

- Since the behavior of multiple read( ) operations on a single mouse instance is undefined, the driver will limit the I/O system to only one open file descriptor per mouse instance.
- The mouse driver shall support VxWorks select( ) functionality.
- To support the application programmer, the driver shall have ioctl( ) functions that will return the number of available characters.
- The mouse device is an HID device, which has support for **Set_Idle** and **Get_Idle** requests. The class driver should export ioctl( ) interfaces to allow user applications to perform these operations.

The design decision for this example is to implement the mouse class driver as a simple character driver. A character driver uses the conventional C programming I/O facilities, such as read( ) and ioctl( ), and allows the programmer to define methods to open( ), close( ), read( ), write( ), create( ), remove( ), and ioctl( ). However, for the mouse class driver, the device is created and removed when the *addDevice* and *removeDevice* callbacks are invoked; and it does not make sense to write to the mouse device. Consequently, those I/O calls are left undefined for this driver. So, the methods to implement are:

- open( )
- read( )
- close( )
- ioctl( )

Consider Asynchronous Device Insertion and Removal

Handling asynchronous insertion and removal of devices is a challenging part of writing a USB driver. For this operation, first evaluate the characteristics that are required.

Recall that, when a device is attached, the *addDevice* callback is invoked, and the device descriptors have been parsed and the device data structure is created. Furthermore, a device instance may be installed into the other VxWorks system components—such as for the USB storage device, which may require a file system XBD instance to interface with file system layer.

If a device is removed, the following actions should be taken:

1. All outstanding read( ) operations should return EOF, which is defined in VxWorks as a **-1**.
2. Any subsequent open( ) or ioctl( ) calls should return a failure.
3. A close( ) on an open file descriptor should succeed.

The challenging part comes when the device is removed asynchronously from the system and the *removeDevice* callback is invoked. The driver must properly lock and synchronize mechanisms to safely remove or delete these device instances from the system. For this example mouse class driver, this is implemented using the following procedures:

1. Create a class driver specific data structure of type **USB2_MSE_DEVICE**.
2. Install an instance into the VxWorks **iosLib** to provide basic POSIX API interface to meet the driver requirement.
3. Link the **USB2_MSE_DEVICE** instance to a driver global device list to keep track of how many USB mice have been attached to the system and what device instance name is available for the next instance connected.
4. Protect the list operations by a class driver mutex.
5. When the mouse is removed, delete the instance from the list.
6. Remove the device instance from the VxWorks **iosLib**, so that further application I/O operations to the opened file descriptor are discarded by the **iosLib**, and fail gracefully.

So, the first step is to create the device instance, documented in Create a Generic Class Device Structure on page 29.

6. Create a Generic Class Device Structure

The next step in implementing the US mouse class driver *addDevice* callback is to create a generic class driver structure for the new device.

About This Task

USB2_CLASS_DEVICE is a generic structure representing the class device. This structure provides the following functionality:

- Encapsulates the information that was retrieved during configuration parsing.
- Contains the required resources to perform class- or vendor specific control transfer, including the URB and setup packet buffer, and provides a binary semaphore to notify the control transfer completion.
- Has the building block to link the device into a device list maintained by this driver, so enable iteration across the devices for the driver.

To create the **USB2_CLASS_DEVICE** structure, you use a helper routine, usb2ClassDeviceCreate( ), which also adds the structure into the device list of the class driver **pUsb2MseDriver**. However, because the device list is a global list, you must protect against concurrent access. You can do this by using a driver mutex semaphore. The usb2TakeDriver( ) and usb2ReleaseDriver( ) routines are provided for this purpose.

Procedure

1. Declare and initialize a variable as a pointer to USB 2 class device.

   In the *addDevice* callback, declare this variable as follows:

```
USB2_CLASS_DEVICE *                 pUwsb2ClassDevice = NULL;
```

2. Declare and initialize a variable to hold the current status.

```
STATUS                              status  = ERROR;
```

3. Take the driver mutex, passing the global variable **pUsb2MseDriver**.

```
/* Take the driver mutex before modifying the global list */

status = usb2TakeDriver(pUsb2MseDriver);
```

4. Validate the call.

```
if (OK != status)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): Driver mutex not taken\n",
                         1, 2, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }
```

5. Pass the **pUsb2ClassDevice** variable, just declared, to the create routine.

```
        /*
     * Create the generic class device structure
     * and add it to the device list for this driver
     */

    pUsb2ClassDevice = usb2ClassDeviceCreate (pUsb2MseDriver,
                                              hDevice,
                                              configurationIndex,
                                              bInterfaceNumber,
                                              uAlternateSetting);
```

6. Validate the creation and if it failed, release the mutex.

```
if (NULL == pUsb2ClassDevice)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): usb2ClassDeviceCreate failed\n",
                         1, 2, 3, 4, 5, 6);

        usb2ReleaseDriver(pUsb2MseDriver);

        return USBHST_FAILURE;
        }
```

7. Create a Class Driver-Specific Device Structure

Once you have a generic class device structure, you can continue to create a class driver-specific device structure for the USB mouse. definitions.

About This Task

For this task you need to allocate a **USB2_MSE_DEVICE** structure (USB Mouse Device Structure on page 58), which contains the resources needed to work with the class.

Procedure

1. Declare and initialize a variable as a pointer to USB 2 mouse device structure.

In the *addDevice* callback, declare this variable as follows:

```
USB2_MSE_DEVICE *                    pDevice = NULL;
```

The **USB2_MSE_DEVICE** structure is defined in the **usb2Mse_raw.c** file.

2. Call the helper routine, usb2MseCreate( ) to allocate the mouse device structure.

```
USB2_MSE_VDBG("usb2MseDeviceAdd(): Building and Filling the "
            "Mouse structure, pUsb2ClassDevice %p, uInEndPointAddr %p\n",
            pUsb2ClassDevice,
            uInEndPointAddr, 3, 4, 5, 6);

pDevice =  usb2MseCreate(pUsb2ClassDevice, uInEndPointAddr, wMaxPacketSize);
```

Pass as the second argument, the end point address retrieved in Parse the Configuration Descriptor on page 24.

3. Add debugging code to validate the return value. If it failed, release the mutex.

```
if (NULL == pDevice)
    {
    USB2_MSE_ERR("usb2MseDeviceAdd(): usb2MseCreate failed\n",
                 1, 2, 3, 4, 5, 6);

    usb2ClassDeviceDelete(pUsb2ClassDevice);

    pUsb2ClassDevice = NULL;

    usb2ReleaseDriver(pUsb2MseDriver);

    return USBHST_FAILURE;
    }
```

4. If the creation succeeded, release the mutex.

```
usb2ReleaseDriver(pUsb2MseDriver);
```

8. Configure the Device Mode

This USB mouse class driver example uses the boot protocol for the mouse device; so you must set the device to work in this mode before triggering any actual application data transfer.

Procedure

1. Set the protocol.

According to the HID 1.11 specification, all HID devices default to report the protocol when they are initialized. However, the host should not make any assumptions about the device's state and should set the desired protocol whenever initializing a device. Setting this protocol is done through the **Set_Protocol** request using the helper routine, usb2HidProtoSet( ), which is defined in *installDir* **/pkgs_v2/connectivity/usb/host/class/hid/h/usb2Hid.h**. Be sure to include this header file if you use any of the APIs defined in it.

In the *addDevice* callback, add the following code to set the protocol.

```
    USB2_MSE_VDBG("usb2MseDeviceAdd(): Setting USB2_HID_PROTO_BOOT\n",
                  1, 2, 3, 4, 5, 6);

    hstStatus = usb2HidProtoSet(pDevice->pClassDevice,
                                USB2_HID_PROTO_BOOT,
                                pDevice->deviceTimeout );
```

2. Add debugging code and validate the return value.

```
    if  (USBHST_SUCCESS != hstStatus)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): Could not Set Boot Protocol\n",
                     1, 2, 3, 4, 5, 6);

        usb2MseDelete(pDevice);

        return USBHST_FAILURE;
        }
```

The usb2MseDelete( ) routine is defined in **usb2Mse_raw.c**.

3. Set the idle rate.

The HID specification has a concept of **Set_Idle** request that can be used to set an idle rate of the mouse, which causes the mouse to NAK any polls on an Interrupt IN endpoint while its current report remains unchanged.

If there is no change, polling continues to be NAKed for a given, time-based duration. This is because when the USB mouse is not moved, there is no change in its report data and, thus, no need to report the duplicated result.

So, by default, this mouse class driver sets the idle rate to be **USB2_HID_IDLE_INFINITE** when the mouse device is attached, which means the endpoint inhibits reporting forever, only reporting when a change is detected in the report data.

Setting the idle time is done through the usb2HidIdleTimeSet( ), which is defined in *installDir* **/pkgs_v2/connectivity/usb/host/class/hid/h/usb2Hid.h**.

In the *addDevice* callback, add the following code to set the protocol.

```
/*
     * Set the idle time to infinite, so that the mouse reports a change
     * only when a change does exist
     */

    hstStatus = usb2HidIdleTimeSet(pDevice->pClassDevice,
                                   USB2_HID_IDLE_INFINITE,
                                   pDevice->deviceTimeout);
```

4. Add debugging code and send an information messages.

```
    if (USBHST_SUCCESS != hstStatus)
        {
        USB2_MSE_WARN("usb2MseDeviceAdd(): Could not set idle time, continue\n",
                      1, 2, 3, 4, 5, 6);
    }
```

A failure here can be safely ignored.

9. Add the Device to the I/O System

The final step to complete the *addDevice* callback is to inform the VxWorks I/O system that the device exists.

Procedure

1. Call iosDevAdd( ) to add the device to the I/O system, **iosLib**.

   The parameters for this routine are described in detail in the API Reference: Application Core OS. first two arguments are members of the **pDevice** structure (documented in Create a Class Driver-Specific Device Structure on page 30). The third argument is **usb2MseDrvNum**, which was declared in Write Code for the Initialization Routine on page 3 and is used again when updating the initialization routine (Define the I/O System Methods on page 35).

```
status = iosDevAdd(&pDevice->deviceHeader,
                   pDevice->pClassDevice->pDeviceName,
                   usb2MseDrvNum);
```

2. Add debugging code to validate the return value.

```
if (OK != status)
    {
    USB2_MSE_ERR("usb2MseDeviceAdd(): iosDevAdd failed.\n",
                 1, 2, 3, 4, 5, 6);

    usb2MseDelete(pDevice);

    return USBHST_FAILURE;
    }

USB2_MSE_INFO("Mouse device %s is up and running...\n",
              pDevice->deviceHeader.name, 2, 3, 4, 5, 6);
```

3. Save the device into the driver data pointer.

   Save the new device structure in the **ppDriverData**. This value is later used as a callback parameter to the other device driver callbacks, such *removeDevice*. Then call usbMsgPost( ) to send a message to the USBD.

```
*ppDriverData = pDevice;

usbMsgPost(USBMSG_GEN2_MSE_DEVREADY,
           pDevice,
           pDevice->pClassDevice->pDeviceName);

return USBHST_SUCCESS;
}
```

The device is now known to VxWorks. The **pDevice->pClassDevice->pDeviceName** has been populated when allocating the **USB2_CLASS_DEVICE** for the device, which takes care of generating a unique name for the device. The name is shown if the user issues a devs command at the VxWorks shell.

The mouse class driver device initialization is complete and the device is ready to be polled for data reports. The following sections describe the data transfer operations from a POSIX I/O API perspective.

Results

This task completes the implementation of the USB mouse class driver *addDevice* callback.

3. CREATING THE I/O SYSTEM METHODS

1. Define the I/O System Methods

When implemented as a character driver in VxWorks, the USB class driver must define the methods used by the I/O system.

About This Task

For this example, defining the I/O system is done in the initialization routine, usb2MseInit( ). You add code to the routine that was begun in Write Code for the Initialization Routine on page 3.

Procedure

1. Take the driver mutex passing the global variable **pUsb2MseDriver**.

```
    if (OK != usb2TakeDriver(pUsb2MseDriver))
        {
        USB2_MSE_ERR("usb2MseInit(): Driver cannot be taken\n",
                     1, 2, 3, 4, 5, 6);

        usb2ClassDriverDelete(pUsb2MseDriver);
        pUsb2MseDriver = NULL;

        return ERROR;
        }
```

2. Inform the VxWorks I/O system about the driver.

   Use **usb2MseDrvNum** to install the driver. Recall that the devices serviced by each driver are added to the I/O system with iosDevAdd( ) in the *addDevice* callback in Add the Device to the I/O System on page 33.

   Only the open( ), close( ), read( ), and ioctl( ) routines are defined.

```
/* Tell the vxWorksVxWorks I/O system about this driver */

USB2_MSE_VDBG("usb2MseInit(): Creating the ioLib driver...\n",
                  1, 2, 3, 4, 5, 6);

usb2MseDrvNum = iosDrvInstall ((FUNCPTR) NULL, /* No create routine */
                               (FUNCPTR) NULL, /* No delete routine */
                               (FUNCPTR) usb2MseIosOpen, /* Open routine */
                               (FUNCPTR) usb2MseIosClose,/* Close routine */
                               (FUNCPTR) usb2MseIosRead, /* Read routine */
                               (FUNCPTR) usb2MseIosWrite, /* no write routine */
                               (FUNCPTR) usb2MseIosIoctl);/* ioctl routine */
```

3. Add debugging code to validate the call.

```
if (0 == usb2MseDrvNum )
    {
    USB2_MSE_VDBG("usb2MseInit(): IO Driver not installed\n",
                          1, 2, 3, 4, 5, 6);
```


WNDRVR

```
    usb2ClassDriverDelete(pUsb2MseDriver);

    return USBHST_FAILURE;
    }
```

The usb2ClassDriverDelete( ) is declared in *installDir* **/pkgs_v2/connectivity/usb/host/class/helper/h/usb2Helper.h**.

2. Implement the I/O System Open Routine

Implementing the I/O open( ) routine is a task when writing a USB mouse class driver. This task comprises several steps.

About This Task

This driver example illustrates the design decision to start the actual USB processing when the mouse is opened. That is, the mouse starts accumulating characters as soon as the unit is opened, prior to the posting of a read( ) to the driver.

Declare the Variables and Validate the Arguments on page 36 The first task, when implementing the open( ) routine for the USB mouse class driver, is to declare the variables and validate the arguments.
Create the File Descriptor for the Device on page 38 The next step in implementing the open( ) I/O routine for the USB mouse class driver is to create the file descriptor.
Populate the USB Request Block (URB) on page 39 The next step in implementing the USB mouse class driver open( ) I/O routine is to obtain data from the device.
Submit the URB to the HCD on page 40 Complete the implementation for the USB mouse class driver open( ) I/O routine by submitting the URB to the HCD.
How the Submit Works on page 41 When implementing the USB mouse class driver open( ) I/O routine, you call usbHstURBSubmit( ).

2.1. Declare the Variables and Validate the Arguments

The first task, when implementing the open( ) routine for the USB mouse class driver, is to declare the variables and validate the arguments.

Procedure

1. Use the signature for the open( ) I/O system routine for a USB mouse class driver.

   The entry point to the function is defined by the VxWorks I/O system as follows:

```
LOCAL ULONG usb2MseIosOpen
    (
    DEV_HDR *   pDevHdr,
    char *      notUsed,
    int         flags,
    int         mode
)
```


WNDRVR

The device header that was specified in the iosDevAdd( ) call in the *addDevice* callback (Add the Device to the I/O System on page 33) is passed back to the driver when the open( ) is invoked. For the design of this mouse driver, the remaining arguments passed in by VxWorks are unimportant.

2. Declare local variables for open( ).

   The device header is cast to the correct type. The routine also creates a file descriptor so that the user application can read the mouse data.

```
{
    USB2_MSE_DEVICE *   pDevice = (USB2_MSE_DEVICE * ) pDevHdr;
    USB2_FILE_DESC *    pFd; /* File descriptor structure used */
    USBHST_STATUS       hstStatus;

    USB2_MSE_VDBG ("usb2MseIosOpen(): Opening the Mouse : "
                   "mode %p, flags %p\n", mode, flags, 3, 4, 5, 6);
```

3. Verify these design criteria.

   This driver is based on the following design decisions:

   - This mouse class driver does not support creating a device, since this is not a normal file; rather, it is a special device that is dynamically created when it is attached in the *addDevice* callback and removed by the *removeDevice* callback.
   - It does not make sense to support writes to the mouse device.
   - This mouse class driver allows only one I/O stream per mouse instance; so, if the mouse has already been opened, or the mouse has been removed, then open( ) fails.

     These criteria should be verified.

     1. Take a mutex.

     Acquire the mouse device mutex, which provides exclusive access to the device structure. It also ensures that the device has not been removed . If the mutex call fails, then open( ) fails.

```
    if (OK != usb2TakeDevice (pDevice->pClassDevice))
        {
        USB2_MSE_ERR ("usb2MseIosOpen(): Cannot take"
                          "device ownership\n", 1, 2, 3, 4, 5, 6);

        errnoSet(EINVAL);

        return (ULONG) ERROR;
        }
```

     2. Check the flags and make sure only one user is allowed per device instance.

```
        /*
     * We don't want the device to be created and do not support
     * write to a mouse device, so refuse these flags
     */

    if ((O_RDWR & flags) || (O_WRONLY & flags) || (O_CREAT & flags ))
        {
        USB2_MSE_ERR("usb2MseIosOpen():"
                     "device not writable/creatable \n",
                     1, 2, 3, 4, 5, 6);
```

```
        usb2ReleaseDevice(pDevice->pClassDevice);

        errnoSet(EINVAL);

        return (ULONG) ERROR;
        }
```

3. Verify that there is only one file descriptor being read.

If the **pDevice->pReader** is not NULL, then the device has already been opened, so return an ERROR.

```
    if (NULL != pDevice->pReader)
       {
       USB2_MSE_ERR("usb2MseIosOpen(): Driver only "
                    "allows one reader, another fd is open, return error \n",
                    1, 2, 3, 4, 5, 6);

       usb2ReleaseDevice (pDevice->pClassDevice);

       errnoSet (EINVAL);

       return (ULONG) ERROR;
       }
```

Parent topic: Implement the I/O System Open Routine on page 36

Related information

Create the File Descriptor for the Device on page 38
Populate the USB Request Block (URB) on page 39
Submit the URB to the HCD on page 40
How the Submit Works on page 41

2.2. Create the File Descriptor for the Device

The next step in implementing the open( ) I/O routine for the USB mouse class driver is to create the file descriptor.

Procedure

1. Create a file descriptor structure in the open( ) routine.

```
    pFd = (USB2_FILE_DESC *) OSS_CALLOC (sizeof(USB2_FILE_DESC));
```

2. Add debugging code to validate the file descriptor structure.

```
    if (NULL == pFd)
        {
        USB2_MSE_ERR ("usb2MseIosOpen():Cannot allocate data\n",
                      1, 2, 3, 4, 5, 6);

        usb2ReleaseDevice(pDevice->pClassDevice);

        errnoSet(ENOMEM);
```


WNDRVR

```
        return ERROR;
        }
```

3. Initialize the file descriptor structure.

   The file descriptor saves the flags and the reference to the device.

```
    pFd->pDevice = pDevice;

    pFd->fdType = flags;
```

4. Associate the file descriptor with the device.

   The device structure (**USB2_MSE_DEVICE**) data member for the reader file descriptor is **pReader**. Set this data member to the file descriptor just created; this is essential to indicate that the device is now opened.

```
    pDevice->pReader = pFd;
```

5. Flush the ring buffer of any previous data.

```
    rngFlush(pDevice->hInRingBuffer);
```

   This step ensures that ring buffer is reset to the empty state when the device is opened; because, if successive open( ), read( ), and close( ) routines are called, the ring buffer could contain "garbage" data from the last time the device was opened. See also Implement the I/O System Close Routine on page 47.

Parent topic: Implement the I/O System Open Routine on page 36

Related information

Declare the Variables and Validate the Arguments on page 36
Populate the USB Request Block (URB) on page 39
Submit the URB to the HCD on page 40
How the Submit Works on page 41

2.3. Populate the USB Request Block (URB)

The next step in implementing the USB mouse class driver open( ) I/O routine is to obtain data from the device.

About This Task

After the application calls open( ), it next tries to retrieve data through the file descriptor, using the standard read( ). To inform the HCD that the device driver is ready to accept data, you must fill a USB Request Block (URB). For this, you use helper macros, described in About the URB Fill Macros on page 41.

Recall that the mouse is an Interrupt IN device, so use the **USBHST_FILL_INTERRUPT_URB( )** macro to fill the URB.

```
    USB2_MSE_DBG ("usb2MseIosOpen(): Device set to READ %p \n",
                      1, 2, 3, 4, 5, 6);

    USBHST_FILL_INTERRUPT_URB(pDevice->pInUrb,
                              pDevice->pClassDevice->hDevice,
                              pDevice->bInEndPointAddr,
                              (UCHAR*) pDevice->pReport,
                              sizeof(USB2_MSE_REPORT),
```


WNDRVR

```
                                USBHST_SHORT_TRANSFER_OK,
                                usb2MseInUrbCompleteCallback,
                                (void *) pDevice,
                                USBHST_SUCCESS
                                );
```

Note that usb2MseInUrbCompleteCallback( ) is passed as the URB callback (see Implement the Mouse URB Callback Routine on page 43).

Parent topic: Implement the I/O System Open Routine on page 36

Related information

Declare the Variables and Validate the Arguments on page 36
Create the File Descriptor for the Device on page 38
Submit the URB to the HCD on page 40
How the Submit Works on page 41

2.4. Submit the URB to the HCD

Complete the implementation for the USB mouse class driver open( ) I/O routine by submitting the URB to the HCD.

About This Task

After the URB is filled (as documented in Populate the USB Request Block (URB) on page 39), submit it to the HCD using the USBD routine usbHstURBSubmit( ), which is one of the core routines for the Wind River USB 2.x USBD. This routine abstracts the differences between different host controller drivers, and directs the URB to the appropriate HCD that is responsible for the actual data transfer. It takes as an argument the pointer to the URB structure.

The device structure (**USB2_MSE_DEVICE**) data member **bTransferInCourse** is a boolean that indicates whether the transfer is running. Set it accordingly.

```
    USB2_MSE_VDBG("usb2MseIosOpen(): Submit an URB to start getting data \n",
                    1, 2, 3, 4, 5, 6);

    /* Submitting the URB starts the I/O on this instance of the mouse */

    pDevice->bTransferInCourse = TRUE;

    if ((hstStatus = usbHstURBSubmit(pDevice->pInUrb)) != USBHST_SUCCESS)
        {
        USB2_MSE_ERR("usb2MseIosOpen():  usbHstURBSubmit failed %d\n",
                    hstStatus, 2, 3, 4, 5, 6);

        errnoSet (EIO);

        pDevice->bTransferInCourse = FALSE;

        pDevice->pReader = NULL;

        OSS_FREE (pFd);

        usb2ReleaseDevice(pDevice->pClassDevice);

        return (ERROR);
        }
```

```
usb2ReleaseDevice(pDevice->pClassDevice);

return (ULONG) pFd;
}
```

The implementation for the open( ) routine is now complete. The routine returns the pointer to the file descriptor structure that was created in Create the File Descriptor for the Device on page 38. This return value is passed back to the driver by the VxWorks I/O system when the various I/O methods, such as read( ), are invoked.

At this point, the driver is capable of receiving characters. The URB callback routine usb2MseInUrbCompleteCallback( ), as specified in the URB (Populate the USB Request Block (URB) on page 39), is invoked by the HCD when either the URB completed or an error occurred, such as the device being removed.

Parent topic: Implement the I/O System Open Routine on page 36

Related information
Declare the Variables and Validate the Arguments on page 36
Create the File Descriptor for the Device on page 38
Populate the USB Request Block (URB) on page 39
How the Submit Works on page 41

2.5. How the Submit Works

When implementing the USB mouse class driver open( ) I/O routine, you call usbHstURBSubmit( ).

When the URB is submitted (Submit the URB to the HCD on page 40), the HCD periodically polls the device's specified endpoint at the specified interval (in the endpoint descriptor **bInterval** field) until either the device has data to deliver or the URB is cancelled.

Note: It is actually the interrupt transfer type (defined in USB) that is doing the polling.

Be aware that usbHstURBSubmit( ) only submits the work to be queued up by the HCD. It does not block to wait for the data to actually come back. When the HCD gets data from the device, it stops further polling and notifies the class driver using the URB callback asynchronously (see Implement the Mouse URB Callback Routine on page 43).

Parent topic: Implement the I/O System Open Routine on page 36

Related information
Declare the Variables and Validate the Arguments on page 36
Create the File Descriptor for the Device on page 38
Populate the USB Request Block (URB) on page 39
Submit the URB to the HCD on page 40

3. About the URB Fill Macros

Because the URB is a complex data structure, and not all fields are used in each transfer type, the USB stack provides a set of macros to facilitate using it.

The macros used for filling the appropriate fields of the URB data structure are defined in *installDir* **/pkgs_v2/connectivity/usb/core/h/usbHst.h**.

USBHST_FILL_CONTROL_URB( )

prepare for control URB

USBHST_FILL_BULK_URB( )

prepare for bulk URB

URB USBHST_FILL_BULK_STREAM_URB( )

prepare for bulk stream URB

USBHST_FILL_INTERRUPT_URB( )

prepare for interrupt URB

USBHST_FILL_ISOCHRONOUS_URB( )

prepare for isochronous URB

The **USBHST_FILL_INTERRUPT_URB( )** macro, in particular, is used in the open( ) and read( ) I/O system methods, to fill the URB to get mouse data. For details, see Populate the USB Request Block (URB) on page 39 and Implement the I/O System Read Routine on page 48.The macro definition is shown below.

```
/*****************************************************************************
 * Macro Name  : USBHST_FILL_INTERRUPT_URB
 * Description : Initialize the URB with requisite parameters for interrupt
 *               transfer.
 * Parameters  : Urb                   OUT Pointer to URB
 *               Device                IN  Handle to the Device
 *               EndPointAddress       IN  End Point Address for which URB is
 *                                          issued.
 *               TransferBuffer        OUT Pointer to the buffer passed as
 *                                          parameter.
 *               TransferLength        IN  Length of the data transfer
 *                                          requested.
 *               TransferFlags         IN  Flags for data transfer.
 *               Callback              IN  Pointer to the callback function.
 *               Context               IN  Class driver specific URB information
 *               Status                OUT Status of the request.
 * Return Type : None
*****************************************************************************/
#define USBHST_FILL_INTERRUPT_URB(Urb,                                      \
                                  Device,                                   \
                                  EndPointAddress,                          \
                                  TransferBuffer,                           \
                                  TransferLength,                           \
                                  TransferFlags,                            \
                                  Callback,                                 \
                                  Context,                                  \
                                  Status)                                   \
{                                                                           \
    /* Store the handle to the Device  */                                   \
    (Urb)->hDevice               = (Device);                                \
                                                                            \
    /* Store the end Point Address for which URB is issued */               \
    (Urb)->uEndPointAddress      = (EndPointAddress);                       \
                                                                            \
    /* Store pointer to the buffer passed as parameter */                  \
    (Urb)->pTransferBuffer       = (TransferBuffer);                        \
```

```
                                                                        \
    /* Store length of the data transfer requested */                   \
    (Urb)->uTransferLength       = (TransferLength);                    \
                                                                        \
    /* Store flags for data transfer */                                 \
    (Urb)->uTransferFlags        = (TransferFlags);                     \
                                                                        \
    /* Store pointer to setup packet */                                 \
    (Urb)->pTransferSpecificData = NULL;                                \
                                                                        \
    /* Store the pointer to the callback function */                    \
    (Urb)->pfCallback            = (Callback);                          \
                                                                        \
    /* Store Class driver specific (Urb) information */                 \
    (Urb)->pContext              = (Context);                           \
                                                                        \
    /* Store the status of the request */                               \
    (Urb)->nStatus               = (Status);                            \
}
```

Some of the parameters require further explanation.

uTransferFlags

This parameter can be one of two values. **USBHST_SHORT_TRANSFER_OK** tells the HCD that it can accept a short transfer; when there is a short transfer, **nStatus** remains as **USBHST_SUCCESS**. **USBHST_START_ISOCHRONOUS_TRANSFER_ASAP** is for isochronous transfer and, thus, is not applicable for the mouse class driver, which uses an interrupt transfer.

pfCallback

This parameter specifies the URB callback when the URB completes or when an error condition is detected by the HCD. The HCD uses it as a function pointer, either when it has successfully transferred the data, or when an error condition occurs and the host cannot continue to transfer the data (such as when the device is removed from the port). The URB callback (Implement the Mouse URB Callback Routine on page 43) is passed a pointer to the URB invoking the callback.

pContext

This parameter is used to save an "opaque parameter" provided by the class driver; when the URB callback is invoked, the class driver casts the **pContext** to a class driver-specific data structure, as a context of the callback. For more information, see Populate the USB Request Block (URB) on page 39 and Implement the Mouse URB Callback Routine on page 43.

4. Implement the Mouse URB Callback Routine

The HCD invokes the URB callback upon the completion of the URB.

About This Task

The URB completion occurs in a VxWorks task context, but at a very high priority level. Consequently, the code in this routine should be succinct. Though not as restrictive as an ISR, if an exception occurs in the URB callback, the entire USB stack crashes. The callback must also handle the asynchronous removal of the device.

Procedure

1. Use the URB callback signature for a USB mouse class driver.

WNDRVR

```
LOCAL USBHST_STATUS usb2MseInUrbCompleteCallback
    (
    pUSBHST_URB pUrb    /* Pointer to the IN URB structure that is completed. */
    )
```

2. Cast the **pContext** to the correct type.

```
    {
    USB2_MSE_DEVICE *   pDevice   = (USB2_MSE_DEVICE* ) pUrb->pContext;
    STATUS              status    = ERROR;
    USBHST_STATUS       hstStatus = USBHST_SUCCESS;
    size_t              nBytes    = 0;
```

3. Validate the arguments.

```
    if ((NULL == pUrb) || (NULL == pUrb->pContext))
        {
        USB2_MSE_ERR("Invalid Parameter\n", 1, 2, 3, 4, 5, 6);
        return USBHST_INVALID_PARAMETER;
        }

    USB2_MSE_VDBG("usb2MseInUrbCompleteCallback(): "
                  "URB returned with result %p\n",
                  pUrb->nStatus, 2, 3, 4, 5, 6);
```

4. Cast the URB's **pContext** member to the class driver-specific data structure.

```
    pDevice   = (USB2_MSE_DEVICE* ) pUrb->pContext;
```

For more information, see Submit the URB to the HCD on page 40 and About the URB Fill Macros on page 41.

5. Take the device mutex.

Call usb2TakeDevice( ) to take the device mutex. For this, also set the device structure (**USB2_MSE_DEVICE**) boolean data member, **bTransferInCourse**, to FALSE, to indicate that it now has no ongoing transfer.

For more information, see About Using Semaphores and Mutexes on page 50.

```
    pDevice->bTransferInCourse = FALSE;

    status = usb2TakeDevice (pDevice->pClassDevice);
```

6. Add debugging code to validate the mutex.

```
    if (ERROR == status)
        {
        USB2_MSE_ERR("usb2MseInUrbCompleteCallback(): Device ownership "
                     "not taken\n",1,2,3,4,5,6);

        return USBHST_FAILURE;
        }
```

7. Check the transfer status.

Check the status of the URB. A URB can be unsuccessful for a variety of reasons, but unless **USBHST_SUCCESS** is returned, the data is invalid.

```
        /*
     * Fill the buffer for what we have been getting, if the URB
     * has been transferred properly. If there is a transfer ERROR
     * with the URB, that means that THIS URB is not proper,
     * so ignore the content
     */

    if ((USBHST_SUCCESS == pUrb->nStatus) &&
        (pUrb->uTransferLength >= sizeof(USB2_MSE_REPORT)))
```

8. Fill the buffer.

   Once the URB completes with **nStatus** of **USBHST_SUCCESS**, the **pDevice->pReport** then points to the data retrieved from the device. The **pDevice->pReport** is specified as the **uTransferBuffer** for the URB. So, the data is put into the ring buffer.

```
        {
        nBytes = rngBufPut(pDevice->hInRingBuffer,
                        (char*) pDevice->pReport,
                        sizeof(USB2_MSE_REPORT));
```

9. Wake up the waiting task.

   If a reader is pending on the device, then wake up the waiting task to get the data from the ring buffer by calling selWakeupAll( ). This was part of the design decision, intended to support the select( ) call on the file descriptor and notify the caller asynchronously.

```
        if (pDevice->pReader)
            {
            if (0 < selWakeupListLen(pDevice->pSelWakeupList) )
                {
                selWakeupAll (pDevice->pSelWakeupList, SELREAD);
                }
            }
```

   The data in this particular driver is a “boot report,” as defined in the HID specification. The task of interpreting this data report is the responsibility of the application of this driver. The class driver's responsibility is to make sure the data is correctly retrieved and sent to the user.

10. Signal the waiting task that the transfer has finished.

    Release the **pDevice->hInSem** binary semaphore, since there may be a reader already pending on this binary semaphore when it called read( ), but there was no data available at the time that read( ) was called.

```
    semGive (pDevice->hInSem);

    USB2_MSE_DBG("usb2MseInUrbCompleteCallback(): Inserted %d bytes"
                    "in the Device Ring buffer, %d bytes available \n",
                    nBytes, rngFreeBytes(pDevice->hInRingBuffer), 3, 4, 5, 6);
    }
```

11. Check whether the file descriptor has been closed.

    Up to here, processing the URB has been done. If there is a reader on this device, the reader may be continuously trying to get the mouse data report. We can choose to submit another URB when the read( ) call is invoked, but we can also continue to submit the next URB in the URB callback itself. In this way, the URB can be reused.

```
        /* If there is no file descriptor, suspend communicating with the device */

    if (NULL == pDevice->pReader)
        {
        USB2_MSE_DBG("usb2MseInUrbCompleteCallback(): File descriptor has "
                     "been closed, return here without submitting \n",
                     1, 2, 3, 4, 5, 6);

        usb2ReleaseDevice(pDevice->pClassDevice);

        return USBHST_SUCCESS;
        }
```

12. Resubmit the URB.

    The code that resubmits the URB is the same as in the open( ) routine (see Submit the URB to the HCD on page 40).

```
        /* Now continue to fill up the URB and re-submit the URB.*/

    USBHST_FILL_INTERRUPT_URB(pDevice->pInUrb,
                      pDevice->pClassDevice->hDevice,
                      pDevice->bInEndPointAddr,
                      (UCHAR*) pDevice->pReport,
                      pDevice->uReportSize,
                      USBHST_SHORT_TRANSFER_OK,
                      usb2MseInUrbCompleteCallback,
                      (void *) pDevice,
                      USBHST_SUCCESS
                      );

    USB2_MSE_VDBG("usb2MseInUrbCompleteCallback(): "
                  "Resubmitting URB...\n", 1, 2, 3, 4, 5, 6);

    /* Prepare requesting data */

    pDevice->bTransferInCourse = TRUE;

    hstStatus = usbHstURBSubmit(pDevice->pInUrb);

    if (USBHST_SUCCESS != hstStatus)
        {
        pDevice->bTransferInCourse = FALSE;

        /* Something wrong with the device */

        USB2_MSE_ERR("usb2MseInUrbCompleteCallback():"
                     "Submitting URB failed : %d\n", hstStatus, 2, 3, 4, 5, 6);
        }
```

13. Once the resubmission completes, the URB callback is done.

```
    USB2_MSE_VDBG("usb2MseInUrbCompleteCallback():  "
                  "URB Submitted with Status %d\n", hstStatus, 2, 3, 4, 5, 6);

    /* And return */

    usb2ReleaseDevice(pDevice->pClassDevice);
```

```
return (hstStatus);
}
```

5. Implement the I/O System Close Routine

The VxWorks I/O system invokes the USB mouse class driver close( ) routine when an application program calls close( ) on a file descriptor associated with the mouse.

Procedure

1. Use the signature for the close( ) I/O system routine for a UBS mouse class driver. For this example, the routine is named usb2MseIosClose( ).

```
LOCAL int usb2MseIosClose
(
USB2_FILE_DESC *    pFd /* File descriptor used by the application */
)
```

2. Cast the file descriptor to the correct type.

   The driver close( ) routine is passed a void value by the VxWorks I/O system, which is the pointer value returned by the driver open( ) routine. In this case, the open( ) routine for the mouse driver returns a pointer to the **USB2_FILE_DESC** structure.

```
{
USB2_MSE_DEVICE *   pDevice =  pFd->pDevice;
```

3. Take the device mutex.

   For more information, see About Using Semaphores and Mutexes on page 50.

```
USB2_MSE_DBG("usb2MseIosClose(): Closing the Mouse %s\n",
              pDevice->deviceHeader.name, 2, 3, 4, 5, 6);

if (OK != usb2TakeDevice (pDevice->pClassDevice))
    {
    USB2_MSE_ERR("usb2MseIosClose():"
                 "Cannot take device ownership\n", 1, 2, 3, 4, 5, 6);

    return ERROR;
    }
```

4. Complete the URB so that it does not resubmit.

   Recall that this driver submits a URB to the USBD on the device open and resubmits it each time the URB completes. There is most likely a URB queued up in the USBD waiting for mouse input. You can ignore the URB and set the **pDevice->pReader** to NULL so that if the URB is completed; it does not resubmit the URB again.

```
if (pDevice->pReader == pFd)
    {
    USB2_MSE_VDBG("usb2MseIosClose(): Reset device reader to NULL.\n",
                  1, 2, 3, 4, 5, 6);
```

```
        pDevice->pReader = NULL;
        }

    /* Cancel the active URB */
    usbHstURBCancel(pDevice->pInUrb);

    OSS_FREE (pFd);

    usb2ReleaseDevice(pDevice->pClassDevice);

    return OK;
}
```

There may be at most one more URB to be completed after the close( ) is called for the device. This is one reason that the rngFlush(pDevice->hInRingBuffer) is necessary to clear the ring buffer when the device is opened. See Create the File Descriptor for the Device on page 38.

6. Implement the I/O System Read Routine

Implementing the read( ) routine for the USB mouse class driver is relatively simple because most of the work is handled in the open( ) and URB callback routines.

Procedure

1. Use the signature for the read( ) I/O system routine for a USB mouse class driver. For this example, the routine is named usb2MseIosRead( ).

```
LOCAL ssize_t usb2MseIosRead
    (
    USB2_FILE_DESC *    pFd,        /* File descriptor used by application */
    char *              pBuffer,    /* Buffer to read to */
    size_t              bfrSize     /* Number of bytes to read */
    )
    {
    USB2_MSE_DEVICE *   pDevice = pFd->pDevice;
    STATUS              status = ERROR;
    size_t              nBytes = 0;

    USB2_MSE_VDBG("usb2MseIosRead(): Start a read %d bytes, to buffer %p\n",
                  bfrSize, (int)pBuffer, 3, 4, 5, 6);
```

2. Take the device mutex.

   Next, take the device mutex to prepare for accessing the ring buffer to check for data availability. For more information, see About Using Semaphores and Mutexes on page 50.

```
    /* Take the device, and check for usability */

    status = usb2TakeDevice (pDevice->pClassDevice);
    if (OK != status)
        {
```


WNDRVR

```
        USB2_MSE_ERR("usb2MseIosRead(): Cannot take device ownership\n",
                     1, 2, 3, 4, 5, 6);

        errnoSet(EIO);

        return ERROR;
        }
```

3. Check whether there is any data in the ring buffer.

   If there is data in the ring buffer, extract all available data, up to the user-specified maximum from the ring buffer, and return it.

```
    /* How much data is already here */

    nBytes = rngNBytes(pDevice->hInRingBuffer);

    USB2_MSE_VDBG("usb2MseIosRead(): We have %d bytes in the buffer\n",
                  nBytes, 2, 3, 4, 5, 6);
```

4. Handle the case of no data.

   If there is no data in the ring buffer, the reader pends until data arrives or an error occurs. So when there is no data available in the ring buffer, the reader task enters the code section to start the wait, pending on the **hInSem** binary semaphore.

   But since the design decision specifies that the driver only allows one reader at a time for a device, you must use a mutex to protect against concurrent reads. For this purpose, the **USB2_MSE_DEVICE** structure for the mouse device has a **hReadMutex** data member.

```
    /* If nBytes is 0, there is nothing in the buffer, so wait for data */

    if (0 == nBytes)
        {
        status = semTake(pDevice->hReadMutex, WAIT_FOREVER);
        if (OK != status)
            {
            USB2_MSE_ERR("usb2MseIosRead(): Cannot take hReadMutex ownership\n",
                         1, 2, 3, 4, 5, 6);

            errnoSet(EIO);

            usb2ReleaseDevice (pDevice->pClassDevice);

            return ERROR;
            }

        /* Release the device mutex so that the URB callback can take it */

        usb2ReleaseDevice (pDevice->pClassDevice);
```

   When the code enters the wait case, call semTake( ) to try to take the **hReadMutex**, so that if another task tries to read the device while a user is pending on this device, it cannot take the **hReadMutex** again.

   Consequently, the **hInSem** is only taken by the previous reader task. When the URB callback is called the **hInSem** is released, so the waiting task can continue to check again for data from the ring buffer.

WNDRVR

```
        /* Do the actual wait */

        status = semTake (pDevice->hInSem, WAIT_FOREVER);

        /* if any error occured during the wait */

        if (OK != status)
            {
            USB2_MSE_ERR("usb2MseIosRead(): Cannot take hInSem ownership\n",
                         1, 2, 3, 4, 5, 6);

            errnoSet(EIO);

            semGive(pDevice->hReadMutex);

            return ERROR;
            }

        semGive(pDevice->hReadMutex);

        status = usb2TakeDevice (pDevice->pClassDevice);

        if (OK != status)
            {
            USB2_MSE_ERR("usb2MseIosRead(): Cannot take device ownership\n",
                         1, 2, 3, 4, 5, 6);

            errnoSet(EIO);

            return ERROR;
            }
        }
```

5. If the buffer has data, copy it to the application buffer.

```
    /* Copy to the application buffer the data we have */

    nBytes = rngBufGet(pDevice->hInRingBuffer, pBuffer, bfrSize);

    usb2ReleaseDevice (pDevice->pClassDevice);

    return (ssize_t)nBytes;
}
```

About Using Semaphores and Mutexes on page 50 The USB mouse class driver uses semaphores and mutexes for locking and synchronizing, so that the reader task does not take control of the whole device.

6.1. About Using Semaphores and Mutexes

The USB mouse class driver uses semaphores and mutexes for locking and synchronizing, so that the reader task does not take control of the whole device.

The order in which calls are made in the read( ) routine is as follows:

```
usb2TakeDevice(pDevice->pClassDevice)
```


WNDRVR

```
semTake(pDevice->hReadMutex,WAIT_FOREVER)
```

```
usb2ReleaseDevice(pDevice->pClassDevice)
```

```
semTake(pDevice->hInSem, WAIT_FOREVER)
```

```
semGive(pDevice->hReadMutex)
```

The semTake(pDevice->hInSem, WAIT_FOREVER) above is a critical operation for reading. While the reader task is waiting for **pDevice->hInSem** to be released (from the URB callback), the code cannot make the device itself taken, otherwise, the URB callback cannot enter the code section to release the **pDevice->hInSem**, because when entering the usb2MseInUrbCompleteCallback( ) the code will also try to take the device. So after we take the **pDevice->hReadMutex**, we have to call usb2ReleaseDevice(pDevice->pClassDevice) to release the device.

By doing this, the URB callback can enter and do something, or the bus monitoring task (see Configuring Bus Tasks) can start deleting procedure by calling usb2MseDeviceRemove( ) which starts by taking the device.

The semaphore and mutex usage strategies are also available to class drivers that provide both read( ) and write( ) functionality. For example, with the printer class driver, you can read and write at the same time. For the mouse class driver, you can read and do ioctl calls at the same time. If you delete the device, however, you have to wait for every semaphore and mutex to be available to really remove the device.

A driver locking mechanism is a very important driver design consideration. Every driver should have a locking mechanism that is appropriate for the class device characteristics. The main goals are:

- Provide global data structure protection.
- Ensure no dead lock condition.
- Ensure no race condition.
- Allow all tasks to operate cooperatively.

These apply to the mouse device class and developers should use them when necessary.

Parent topic: Implement the I/O System Read Routine on page 48

7. Support for the Required ioctl Functionality

Relative to some USB class drivers, the mouse class driver is not complex and requires only a small set of ioctl to be supported.

The required ioctl functionality is implemented in usb2MseIosIoctl( ), which is defined in the **usb2Mse_raw.c** file, which is included in your class driver; see Set Up the Environment and Driver Files on page 1. The codes that are supported are listed below.

FIONREAD

Since a ring buffer was used to save the data reported by the mouse device, user applications may want to know how many bytes are available in that ring buffer to be read out. The **FIONREAD** ioctl is implemented to support this functionality.

FIOSELECT and FIOUNSELECT

As mouse data is reported only when there is mouse movement, user applications may choose to only issue reads when the data is actually available. The standard POSIX way to do this is to use select system calls. The select functionality is implemented in the ioctl with the standard ioctl parameters **FIOSELECT** and **FIOUNSELECT**, by calling selNodeAdd( ) and selNodeDelete( ). See the **selectLib** online documentation for how to implement select support.

USB2_IOCTL_HID_IDLE_TIME_GET

The driver also provides ioctl **USB2_IOCTL_HID_IDLE_TIME_GET** to let the user application get the current idle rate. For information on how the values are to be interpreted, see the HID specification.

USB2_IOCTL_HID_IDLE_TIME_SET

The driver also provides ioctl **USB2_IOCTL_HID_IDLE_TIME_SET** to let the user application set the idle rate. For information on how the values are to be interpreted, see the HID specification.

4. COMPLETING THE REMOVEDEVICE CALLBACK IMPLEMENTATION

1. Complete the removeDevice Callback Implementation

The original *removeDevice* callback, in this USB mouse class driver example, was a placeholder. Update the original placeholder with the fully-implemented version.

About This Task

The USBD calls the registered *removeDevice* routine each time a mouse device is physically removed (see *VxWorks 7 USB Host Stack Programmer's Guide:* Configuring Bus Tasks). This routine is responsible for closing off all input and output that is in progress, and then deleting the data structures associated with the instance of the mouse.

This task can be complex since VxWorks application tasks are outside of the control of the driver and may have references to the data structures used by the instance. Referencing a deleted data structure can have unpredictable results. Keep in mind that the approach to handling this issue for the mouse device driver may not apply exactly to other device drivers.

After handling the I/O mechanism by implementing the necessary I/O system routines and ensuring support for the required iotcl functionality, update the original placeholder callback, created in Implement Simple Driver Callbacks as Placeholders on page 7, with a fully-implemented version. Recall that the operating paradigm for the mouse was defined so that any I/O operation other than close( ) fails after a device is removed.

Procedure

1. Use the same signature.

   Use the same signature that you used for the placeholder. The *removeDevice* callback takes two arguments, the device handle and the pointer to the driver data.

   ```
   LOCAL void usb2MseDeviceRemove
       (
       UINT32  hDevice,     /* Device handler provided by the USBD */
       void *  pDriverData /* Driver structure returned by add routine */
       )
   ```

2. Replace the local variables.

   Replace the local variables in the placeholder version with two new variables, and initialize the **status** variable to ERROR.

   ```
   {
       STATUS              status = ERROR;
       USB2_MSE_DEVICE *   pDevice = pDriverData;
   ```

   The **pDevice** variable is declared as a pointer to a **USB2_MSE_DEVICE** structure, because that is the type that was saved by the *addDevice* callback (and passed in as the **pDriverData** argument). Assign **pDriverData** to **pDevice**.

3. Update the debugging information message.

```
USB2_MSE_DBG("Removing mouse (hDevice 0x%X, pDevice %p)\n",
              hDevice,
              pDevice, 3, 4, 5, 6);

usbMsgPost(USBMSG_GEN2_MSE_DEVREMOVED,
           pDevice,
           pDevice->pClassDevice->pDeviceName);
```

The usbMsgPost( ) routine sends a remove message to the USBD.

4. Take a driver mutex and validate it using the **status** variable.

   Take the driver mutex to prepare to remove a device from the driver's device list, which should be protected from concurrent access.

```
status = usb2TakeDriver(pUsb2MseDriver);
if (OK != status)
    {
    USB2_MSE_ERR("usb2MseDeviceRemove(): Cannot take driver ownership\n",
                 1, 2, 3, 4, 5, 6);
    return;
    }
```

5. Take a device mutex and validate it using the **status** variable.

   For more information on this call, see About Using Semaphores and Mutexes on page 50.

```
status = usb2TakeDevice(pDevice->pClassDevice);
if (OK != status)
    {
    USB2_MSE_ERR("usb2MseDeviceRemove(): "
                 "Cannot take device ownership\n",
                 1, 2, 3, 4, 5, 6);

    usb2ReleaseDriver(pUsb2MseDriver);

    return;
    }
```

6. Send a debugging message specifying which device is being deleted.

```
USB2_MSE_INFO("Delete device %s%d\n",
              pDevice->pClassDevice->pDriver->pDefaultDriverName,
              pDevice->pClassDevice->pNode->uDeviceIndex, 3, 4, 5, 6);
```

7. Delete the device, or send a message if it fails.

```
if (iosDevDelete(&pDevice->deviceHeader) != OK)
    {
    USB2_MSE_WARN("IOS Dev Delete Fail\n",
                  1, 2, 3, 4, 5, 6);
    }
```

8. Send a remove message to the helper task to remove the class driver.

```
usb2DeviceDeleteMsgPost(
    pDevice->pClassDevice,
    usb2MseDeleteCallback,
    pDevice
    );
```

9. Release the driver mutex.

```
/*
 * Release the device mutex since the usb2MseDeleteCallback
 * will need to take the device mutex in order to delete the
 * mutex with respect to the rule "mutex should be taken before
 * delete it"
 */

usb2ReleaseDevice(pDevice->pClassDevice);

usb2ReleaseDriver(pUsb2MseDriver);
}
```

2. Solving a Potential Race Condition

There is a potential race condition that is only an issue for versions of USB prior to VxWorks 6.8.

About This Task

If there is a reader task pending on an open file descriptor, it presents a problem for the driver. The driver cannot delete the structure until the task stops accessing the file descriptor and data structures for the mouse instance. For example, consider one case where the usb2MseDeviceRemove( ) is called from the Bus Monitoring task (see Configuring Bus Tasks), and the reader task is still pending on the **pDevice->hInSem** in the usb2MseIosRead( ) routine. If we delete the device structure of the mouse instance, there will be an invalid reference in the reader task and it would cause unpredictable results. In this case, there is still one URB not completed through the callback and the reader task is actually pending on that URB. In short, if there is any URB not completed when the *removeDevice* callback is called, there would be a race condition that may cause invalid structure references.

To solve the problem stated above, one simple decision is to let the usb2MseDeviceRemove( ) to wait until the URBs complete and no other tasks are referring to the device data structures. But as we have stated, the usb2MseDeviceRemove( ) is called from the USB Bus Monitoring task (see Configuring Bus Tasks), which cannot allow the user application or the URB completion tasks to wait too long, because this would in essence hang the USB device or negatively affect the user experience. So in this case, a helper task is spawned in the GEN2 class driver helper module. When we want to delete a device structure, we post a delete message to that dedicated task, which will try to call the device delete callback routine usb2MseDeleteCallback( ) periodically in a task context that is different with the USB Bus Monitoring task (see Configuring Bus Tasks). The usb2MseDeleteCallback( ) will check the URB in progress status flag **pDevice->bTransferInCourse**, and if it is set to FALSE, then it knows there are no URBs pending, and can proceed with the device deletion.

VxWorks 6.8 and Later USB Stack

The scenario described above is not the case for the VxWorks 6.8 and later USB stack, as the stack now has the following sequence of events that happen when a device is removed:

- Delete the USB pipes of the device.
- Call the class driver *removeDevice* callback.

While deleting the USB pipes of the device, the USBD will cancel all the pending URBs on these pipes, and the canceled URBs will call the URB callback when it is reclaimed by the HCD. This is to say, when the *removeDevice* callback routine is called, the callbacks of the pending URBs should have been called from the HCD URB reclamation task. And since the pipes have been deleted, there will be no further URBs to be submitted in the URB callbacks themselves. So in short, when the *removeDevice* callback is called, the class driver can consider it safe that there are no incomplete URBs.

However, to leverage the usage of the GEN2 class drivers on top of all possible HCD implementations (including former Wind River USB stack HCDs and customer designed HCDs), the class driver should take the case described above into account. That is, when the *removeDevice* callback routine is called, there may still incomplete URBs, and thus there may be user tasks referring to the device data structures.

5. COMPILING THE DRIVER FILES

1. Compile the Driver Files

The final step in creating a USB mouse class driver is to compile the files.

About This Task

When the driver infrastructure is complete, compile the source files in *installDir* **/pkgs_v2/connectivity/usb/host/class/mouse**. Also include the USB library in your VxWorks image. You can then call the driver initialization routines in your code at the proper stage of system initialization. For more information, see Compile the USB Mouse Class Driver and Test It on page 8.

Note: You can optionally add the components to the CDF file. For details on creating CDF files, see the VxWorks 7 Custom Component and CDF Developer's Guide.

6. REFERENCE OF STRUCTURES, CALLBACKS, AND ROUTINES

1. Standard Configuration Descriptor Structure

This structure is used in the *addDevice* callback for the USB mouse class driver.

This structure is defined as **pUSBHST_CONFIG_DESCRIPTOR** in **usbHst.h** and is used to obtain information in the addDevice( ) callback. For more information, see Set the Configuration on page 18.

```
typedef struct usbhst_config_descriptor
    {
    UINT8  bLength;             /* Size of this descriptor in bytes          */
    UINT8  bDescriptorType;     /* CONFIGURATION Descriptor Type             */
    UINT16 wTotalLength;        /* Configuration data length                 */
    UINT8  bNumInterfaces;      /* Number of interfaces supported            */
    UINT8  bConfigurationValue; /* SetConfiguration() function argument      */
    UINT8  iConfiguration;      /* Configuration string descriptor           */
    UINT8  bmAttributes;        /* Bitmap configuration characteristics      */
    UINT8  MaxPower; /* Max pwr consumption (mA) of operational device on bus*/
    }__attribute__((packed)) USBHST_CONFIG_DESCRIPTOR,
    *pUSBHST_CONFIG_DESCRIPTOR;
```

2. USB Mouse Device Structure

This structure is used in the *addDevice* callback for the USB mouse class driver.

The **USB2_MSE_DEVICE** structure is defined below and is used in the *addDevice* callback. For more information, see Create a Class Driver-Specific Device Structure on page 30.

```
typedef struct usb2_mse_device
    {
    DEV_HDR                 deviceHeader;       /* I/O system device header */
    USB2_CLASS_DEVICE *     pClassDevice;       /* Generic class device info */
    SEM_ID                  hInSem;             /* signaling sem, in transfer*/
    SEM_ID                  hReadMutex;         /* Protect concurrent read */
    USBHST_URB *            pInUrb;             /* In transfer URB */
    UINT32                  uInBufferNbElem;    /* In ring buffer elements */
    RING_ID                 hInRingBuffer;      /* In ring buffer handle */
    UINT8                   bInEndPointAddr;    /* In endpoint addr */
    int                     deviceTimeout;      /* Timeout (ms) to wait reqs. */
    USB2_FILE_DESC *        pReader;            /* Reader file descriptor */
    USB2_MSE_REPORT *       pReport;            /* Report structure read */
    UINT8                   uIdleTime;          /* Idle time */
    BOOL                    bTransferInCourse;  /* Is a transfer running? */
    SEL_WAKEUP_LIST *       pSelWakeupList;     /* Select wakeup list */
    BOOL                    bSelListInitialized; /* above list initialized */
    } USB2_MSE_DEVICE;
```

Type	Value	Description
deviceHeader		This is a structure that is used by the VxWorks I/O system and contains information used by VxWorks **iosLib**. It should not be altered by the device driver itself.
pClassDevice		This is a pointer to the generic class device structure, and is referenced to perform class-specific requests.
bInEndPointAddr, hInSem, pInUrb, pReport, bTransferInCourse		These members are resources used to do the data transfer.
uInBufferNbElem, hInRingBuffer, hReadMutex, pReader		These members support **iosLib** for reading data.
pSelWakeupList, bSelListInitialized		These members are the resources that provide select support of the driver.
deviceTimeout, idleTime		These members are timeout values for the device.

3. Complete addDevice Callback

The *addDevice* callback for this USB mouse class driver example is called usb2MseDeviceAdd( ).

The parameters to the usb2MseDeviceAdd( ) callback are described below.

hDevice

Specifies a device handle, generated by the USBD, that is the central point that you can use to communicate with the USBD APIs. Class drivers should treat it as an opaque handle. There is one important **USBD_DEVICE_INFO** structure associated with this handle (see Set the Configuration and Interface on page 15).

bInterfaceNumber

Identifies the interface by number in order to distinguish between different interfaces in the same configuration. To support USB composite devices, which have several interfaces in the same configuration, USBD tries to call the *addDevice* callback for each interface. [1] on page 64

uSpeed

Specifies the speed at which information is found by the host controller driver. From a class driver's perspective, it is not important other than to show the speed.

ppDriverData

Specifies the output parameter, which is of type void **. Normally, you should create a data structure to represent the device and its associated resources. When the *addDevice* callback is called and passed the various class driver verifications, and you want to take this device, you can allocate such a data structure, and save the pointer to that data structure in the ppDriverData parameter. The USBD saves the pointer saved in this parameter; and when it calls the other callbacks it passes this pointer as a parameter to them, so that the callbacks can cast to the class driver-defined device structure, and then work with it.

The complete usb2MseDeviceAdd( ) callback routine is shown below.

```
LOCAL USBHST_STATUS usb2MseDeviceAdd
    (
    UINT32  hDevice,                /* device handler returned by usbd */
    UINT8   bInterfaceNumber,       /* interface number found by the usbd */
    UINT8   uSpeed,                 /* speed of the device */
    void ** ppDriverData            /* place to put the device data*/
    )
    {
    USB2_MSE_DEVICE *                   pDevice = NULL;
    STATUS                              status  = ERROR;
    USBHST_STATUS                       hstStatus = USBHST_FAILURE;
    pUSBD_INTERFACE_INFO                pInterface = NULL;
    pUSBD_DEVICE_INFO                   pDeviceInfo = NULL;
    pUSBHST_CONFIG_DESCRIPTOR           pCurrentConfiguration = NULL;
    USB2_CLASS_DEVICE *                 pUsb2ClassDevice = NULL;
    UINT8                               uInEndPointAddr = 0;
    UINT8                               configurationIndex = 0;
    UINT8                               uAlternateSetting = 0;
    USBHST_INTERFACE_DESCRIPTOR *       pInterfaceDesc;
    USBHST_ENDPOINT_DESCRIPTOR *        pEndPointDesc;
    UINT16                              wMaxPacketSize;
    USB_TRANSFER_SETUP_INFO             xferSetup;

    USB2_MSE_INFO("Attaching mouse "
                  "(hDevice 0x%X, interface %d, speed %d)\n",
                  hDevice, bInterfaceNumber, uSpeed, 4, 5, 6);

    /* Get the device information, for the descriptors */

    usbdTranslateDeviceHandle(hDevice, &pDeviceInfo);

    if (NULL == pDeviceInfo )
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): Mouse is apparently "
                     "not connected anymore, the USBD pDeviceInfo is NULL\n",
                      1, 2, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }

    pCurrentConfiguration = (pUSBHST_CONFIG_DESCRIPTOR)
                                      pDeviceInfo->pCurrentConfiguration;
    if (NULL == pCurrentConfiguration)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): pCurrentConfiguration NULL\n",
                     1, 2, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }

    configurationIndex = (UINT8)(pCurrentConfiguration->bConfigurationValue -1);

    if (usbHstSetConfiguration (hDevice,configurationIndex ) != OK)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): Failed to set configuration - "
```

```
                  "device not added.\n", 1, 2, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }

    pInterface = &(pDeviceInfo->pInterfacesInfo[bInterfaceNumber]);

    if (NULL == pInterface)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd():No Interface %d\n",
                     bInterfaceNumber, 2, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }

    uAlternateSetting = pInterface->uAlternateSetting;

    USB2_MSE_VDBG("usb2MseDeviceAdd(): Setting Interface %d \n",
                     bInterfaceNumber, 2, 3, 4, 5, 6);


    if  ((hstStatus = usbHstSetInterface(hDevice,
                                         bInterfaceNumber,
                                         uAlternateSetting))
          != USBHST_SUCCESS)
         {
         USB2_MSE_ERR("usb2MseDeviceAdd(): Failed to set interface - "
                     "device not added \n", 1, 2, 3, 4, 5, 6);

         /*
          * If a device only supports a default setting for the specified
          * interface, then a STALL may be returned in the Status stage
          * of the request. And in this case, continue, since this is a
          * control pipe protocol STALL and will be cleared automatically
          * when the device received the next setup packet.
          */

         if (hstStatus != USBHST_STALL_ERROR)
             return USBHST_FAILURE;
         }

    USB2_MSE_VDBG("usb2MseDeviceAdd(): Searching for endpoints... \n",
                      1, 2, 3, 4, 5, 6);

    pInterfaceDesc = usb2FindInterface(pDeviceInfo,
                                       bInterfaceNumber,
                                       uAlternateSetting);
    if (NULL == pInterfaceDesc)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): No such interface"
                     " bInterfaceNumber %d, uAlternateSetting %d on device\n",
                     bInterfaceNumber, uAlternateSetting, 3, 4, 5, 6);

        return USBHST_FAILURE;
        }

    /* Find the must-have Interrupt IN endpoint */
```

```
pEndPointDesc = usb2FindEndpoint((UCHAR* )pInterfaceDesc,
                                  USB2_ATTR_INTERRUPT,
                                  USB2_ENDPOINT_IN,
                                  0,
                                  0,
                                  0);
if (NULL == pEndPointDesc)
    {
    USB2_MSE_ERR("usb2MseDeviceAdd(): NO IN endpoint found\n",
                 1, 2, 3, 4, 5, 6);

    return USBHST_FAILURE;
    }

uInEndPointAddr = pEndPointDesc->bEndpointAddress;

wMaxPacketSize = OS_UINT16_LE_TO_CPU(pEndPointDesc->wMaxPacketSize);

/* Set up the transfer requirment */

OS_MEMSET(&xferSetup, 0, sizeof(USB_TRANSFER_SETUP_INFO));

xferSetup.uMaxTransferSize = max(_CACHE_ALIGN_SIZE, wMaxPacketSize);
xferSetup.uMaxNumReqests = 1;
xferSetup.uFlags = 0;

hstStatus = usbHstPipePrepare(hDevice,
                pEndPointDesc->bEndpointAddress,
                &xferSetup);

if (hstStatus != USBHST_SUCCESS)
    {
    USB2_MSE_ERR("Interrupt IN at 0x%x set up failed\n",
        pEndPointDesc->bEndpointAddress, 2, 3, 4, 5, 6);

    return USBHST_FAILURE;
    }

status = usb2TakeDriver(pUsb2MseDriver);
if (OK != status)
    {
    USB2_MSE_ERR("usb2MseDeviceAdd(): Driver mutex not taken\n",
                     1, 2, 3, 4, 5, 6);

    return USBHST_FAILURE;
    }

USB2_MSE_VDBG("usb2MseDeviceAdd(): Building and Filling the "
        "Generic Device structure, driver %p, hDevice 0x%X, config 0x%X, "
        "interface 0x%X, altSetting 0x%X... \n",
        pUsb2MseDriver,
        hDevice,
        configurationIndex,
        bInterfaceNumber,
        uAlternateSetting, 6);
```

```
pUsb2ClassDevice = usb2ClassDeviceCreate (pUsb2MseDriver,
                                          hDevice,
                                          configurationIndex,
                                          bInterfaceNumber,
                                          uAlternateSetting);
if (NULL == pUsb2ClassDevice)
    {
    USB2_MSE_ERR("usb2MseDeviceAdd(): usb2ClassDeviceCreate failed\n",
                     1, 2, 3, 4, 5, 6);

    usb2ReleaseDriver(pUsb2MseDriver);

    return USBHST_FAILURE;
    }

USB2_MSE_VDBG("usb2MseDeviceAdd(): Building and Filling the "
            "Mouse structure, pUsb2ClassDevice %p, uInEndPointAddr %p"
            "wMaxPacketSize %x\n", pUsb2ClassDevice,
            uInEndPointAddr, wMaxPacketSize, 4, 5, 6);

pDevice =  usb2MseCreate(pUsb2ClassDevice, uInEndPointAddr, wMaxPacketSize);

if (NULL == pDevice)
    {
    USB2_MSE_ERR("usb2MseDeviceAdd(): usb2MseCreate failed\n",
                 1, 2, 3, 4, 5, 6);

    usb2ClassDeviceDelete(pUsb2ClassDevice);

    pUsb2ClassDevice = NULL;

    usb2ReleaseDriver(pUsb2MseDriver);

    return USBHST_FAILURE;
    }

usb2ReleaseDriver(pUsb2MseDriver);

USB2_MSE_VDBG("usb2MseDeviceAdd(): Setting USB2_HID_PROTO_BOOT\n",
              1, 2, 3, 4, 5, 6);

hstStatus = usb2HidProtoSet(pDevice->pClassDevice,
                            USB2_HID_PROTO_BOOT,
                            pDevice->deviceTimeout );

if (USBHST_SUCCESS != hstStatus)
    {
    USB2_MSE_ERR("usb2MseDeviceAdd(): Could not Set Boot Protocol\n",
                 1, 2, 3, 4, 5, 6);

    usb2MseDelete(pDevice);

    return USBHST_FAILURE;
    }

/*
 * Set the idle time to infinite, so that the mouse reports a change
```

```
     * only when a change does exist
     */

    hstStatus = usb2HidIdleTimeSet(pDevice->pClassDevice,
                                   USB2_HID_IDLE_INFINITE,
                                   pDevice->deviceTimeout);

    if (USBHST_SUCCESS != hstStatus)
        {
        USB2_MSE_WARN("usb2MseDeviceAdd(): Could not set idle time, continue\n",
                      1, 2, 3, 4, 5, 6);

        /* Go and ignore this as it is acceptable for this failure */
        }

    USB2_MSE_VDBG("usb2MseDeviceAdd(): Adding device %s to IO driver %d\n",
                                  pDevice->pClassDevice->pDeviceName,
                                  usb2MseDrvNum,
                                  3, 4, 5, 6);

    status = iosDevAdd(amp;&pDevice->deviceHeader,
                       pDevice->pClassDevice->pDeviceName,
                       usb2MseDrvNum);

    if (OK != status)
        {
        USB2_MSE_ERR("usb2MseDeviceAdd(): iosDevAdd failed.\n",
                     1, 2, 3, 4, 5, 6);

        usb2MseDelete(pDevice);

        return USBHST_FAILURE;
        }

    USB2_MSE_INFO("Mouse device %s is up and running...\n",
                  pDevice->deviceHeader.name, 2, 3, 4, 5, 6);

    *ppDriverData = pDevice;

    usbMsgPost(USBMSG_GEN2_MSE_DEVREADY,
               pDevice,
               pDevice->pClassDevice->pDeviceName);

    return USBHST_SUCCESS;
    }
```

[1] on page 59

Most device classes use a single configuration with a single interface. If you are writing a driver that has several interfaces for different functionality, you should check the bInterfaceNumber to make sure it is the correct interface number that your driver is responsible for handling. If it is not the one you are expecting (but is expected by another driver), you should return with a failure here.

If you are writing a driver that is to work with a device with several interfaces, when the USBD calls the *addDevice* callback for the first interface, you should take this interface and ignore the other interface numbers.

WNDRVR

4. Complete removeDevice Callback

The *removeDevice* callback for this USB mouse class driver example is called usb2MseDeviceRemove( ).

This routine is called by the USBD each time a device is removed from the system. This routine is responsible for ensuring all outstanding IO requests are completed or aborted and tearing down the I/O structure.

The complete example for the *removeDevice* callback is shown below.

```
LOCAL void usb2MseDeviceRemove
    (
    UINT32  hDevice,    /* Device handler provided by the USBD */
    void *  pDriverData /* Driver structure returned by add routine */
    )
    {
    STATUS              status = ERROR;
    USB2_MSE_DEVICE *   pDevice = pDriverData;

    USB2_MSE_DBG("Removing mouse (hDevice 0x%X, pDevice %p)\n",
                 hDevice,
                 pDevice, 3, 4, 5, 6);

    usbMsgPost(USBMSG_GEN2_MSE_DEVREMOVED,
               pDevice,
               pDevice->pClassDevice->pDeviceName);

    status = usb2TakeDriver(pUsb2MseDriver);
    if (OK != status)
        {
        USB2_MSE_ERR("usb2MseDeviceRemove(): Cannot take driver ownership\n",
                     1, 2, 3, 4, 5, 6);
        return;
        }

    status = usb2TakeDevice(pDevice->pClassDevice);
    if (OK != status)
        {
        USB2_MSE_ERR("usb2MseDeviceRemove(): "
                     "Cannot take device ownership\n",
                     1, 2, 3, 4, 5, 6);

        usb2ReleaseDriver(pUsb2MseDriver);

        return;
        }

    if (pDevice->bSelListInitialized == TRUE)
        {
        selWakeupListTerm(pDevice->pSelWakeupList);

        pDevice->bSelListInitialized = FALSE;
        }

    USB2_MSE_INFO("Delete device %s%d\n",
                  pDevice->pClassDevice->pDriver->pDefaultDriverName,
                  pDevice->pClassDevice->pNode->uDeviceIndex, 3, 4, 5, 6);
```

```
    if (iosDevDelete(&pDevice->deviceHeader) != OK)
        {
        USB2_MSE_WARN("IOS Dev Delete Fail\n",
                      1, 2, 3, 4, 5, 6);
        }

    usb2DeviceDeleteMsgPost(
        pDevice->pClassDevice,
        usb2MseDeleteCallback,
        pDevice
        );

    /*
     * Release the device mutex since the usb2MseDeleteCallback
     * will need to take the device mutex in order to delete the
     * mutex with respect to the rule "mutex should be taken before
     * delete it"
     */

    usb2ReleaseDevice(pDevice->pClassDevice);

    usb2ReleaseDriver(pUsb2MseDriver);
    }
```

5. Complete Initialization Routine

This routine is called once to register the mouse class driver with the USBD. While multiple instances of the mouse are supported, the driver is initialized only once.

For this example, the initialization routine is named usb2MseInit( ). The complete routine is shown below. For more information, see Write Code for the Initialization Routine on page 3.

```
STATUS usb2MseInit
    (
    char *  pName /* Default base name that will be used, or NULL if no names */
    )
    {
    USBHST_STATUS   hstStatus;

    USB2_MSE_VDBG("usb2MseInit(): Initializing the mouse driver...\n",
                  1, 2, 3, 4, 5, 6);

    /* If already initialized, ignore and return. */

    if (NULL != pUsb2MseDriver)
        {
        USB2_MSE_WARN("usb2MseInit(): Driver already initialized, return...\n",
                      1, 2, 3, 4, 5, 6);
        return OK;
        }

    /* If no valid parameter is given, take the driver default name. */

    if (NULL == pName)
```

```
        pUsb2MseDriver = usb2ClassDriverCreate (USB2_MSE_DEVICE_BASE_NAME);
    else
        pUsb2MseDriver = usb2ClassDriverCreate (pName);

    /* If the driver cannot be created, return with ERROR. */

    if (NULL == pUsb2MseDriver)
        {
        USB2_MSE_ERR("usb2MseInit(): Driver cannot be created\n",
                      1, 2, 3, 4, 5, 6);
        return ERROR;
        }

    if (OK != usb2TakeDriver(pUsb2MseDriver))
        {
        USB2_MSE_ERR("usb2MseInit(): Driver cannot be taken\n",
                      1, 2, 3, 4, 5, 6);

        usb2ClassDriverDelete(pUsb2MseDriver);
        pUsb2MseDriver = NULL;

        return ERROR;
        }

    USB2_MSE_VDBG("usb2MseInit(): Creating the ioLib driver...\n",
                  1, 2, 3, 4, 5, 6);

    usb2MseDrvNum = iosDrvInstall (
                        (DRV_CREATE_PTR) NULL,      /* No create routine */
                        (DRV_REMOVE_PTR) NULL,      /* No delete routine */
                        (DRV_OPEN_PTR) usb2MseIosOpen,    /* Open routine */
                        (DRV_CLOSE_PTR) usb2MseIosClose,  /* Close routine */
                        (DRV_READ_PTR) usb2MseIosRead,    /* Read routine */
                        (DRV_WRITE_PTR) usb2MseIosWrite,  /* no write routine */
                        (DRV_IOCTL_PTR) usb2MseIosIoctl); /* ioctl routine */

    if (0 == usb2MseDrvNum )
        {
        USB2_MSE_VDBG("usb2MseInit(): IO Driver Not installed\n",
                       1, 2, 3, 4, 5, 6);

        usb2ClassDriverDelete(pUsb2MseDriver);
        pUsb2MseDriver = NULL;

        return USBHST_FAILURE;
        }

    pUsb2MseDriver->hstDriver.bFlagVendorSpecific = FALSE;
    pUsb2MseDriver->hstDriver.uVendorIDorClass = USB2_HID_CLASS;
    pUsb2MseDriver->hstDriver.uProductIDorSubClass = USBD_NOTIFY_ALL;
    pUsb2MseDriver->hstDriver.uBCDUSBorProtocol = USB2_MSE_PROTOCOL;
    pUsb2MseDriver->hstDriver.addDevice = usb2MseDeviceAdd;
    pUsb2MseDriver->hstDriver.removeDevice = usb2MseDeviceRemove;
    pUsb2MseDriver->hstDriver.suspendDevice =  usb2MseSuspendResume;
    pUsb2MseDriver->hstDriver.resumeDevice = usb2MseSuspendResume;

    USB2_MSE_VDBG("usb2MseInit(): Registering the USBD Part:\n",
```

```
                    1, 2, 3, 4, 5, 6);
    USB2_MSE_VDBG("usb2MseInit(): Vendorspecific : 0x%X :\n",
            pUsb2MseDriver->hstDriver.bFlagVendorSpecific, 2, 3, 4, 5, 6);

    USB2_MSE_VDBG("usb2MseInit(): uVendorIDorClass : 0x%X :\n",
            pUsb2MseDriver->hstDriver.uVendorIDorClass, 2, 3, 4, 5, 6);

    USB2_MSE_VDBG("usb2MseInit(): uProductIDorSubClass : 0x%X :\n",
            pUsb2MseDriver->hstDriver.uProductIDorSubClass, 2, 3, 4, 5, 6);

    USB2_MSE_VDBG("usb2MseInit(): uBCDUSBorProtocol : 0x%X :\n",
            pUsb2MseDriver->hstDriver.uBCDUSBorProtocol, 2, 3, 4, 5, 6);

    /* Register the client with USBD */

    USB2_MSE_VDBG("usb2MseInit(): registering the client with USBD...\n",
                    1, 2, 3, 4, 5, 6);

    hstStatus = usbHstDriverRegister (&pUsb2MseDriver->hstDriver,
                                      NULL, "usbGen2Mouse");

    if (hstStatus != USBHST_SUCCESS)
        {
        USB2_MSE_ERR("usb2MseInit(): cannot register with USBD...\n",
                    1, 2, 3, 4, 5, 6);

        iosDrvRemove(usb2MseDrvNum, TRUE);
        usb2MseDrvNum = 0;

        usb2ClassDriverDelete(pUsb2MseDriver);
        pUsb2MseDriver = NULL;

        return ERROR;
        }

    USB2_MSE_DBG("usb2MseInit(): Class driver registration succeeded.\n",
                  1, 2, 3, 4, 5, 6);

    usb2ReleaseDriver(pUsb2MseDriver);

    return OK;
    }
```

6. Complete URB Callback Routine

The URB callback for this USB mouse class driver example is called usb2MseInUrbCompleteCallback( ).

The complete URB callback routine, usb2MseInUrbCompleteCallback( ), is shown below. For more information about this routine, see Implement the Mouse URB Callback Routine on page 43.

```
LOCAL USBHST_STATUS usb2MseInUrbCompleteCallback
    (
    pUSBHST_URB pUrb    /* Pointer to the IN URB structure that is completed. */
    )
```

```
{
USB2_MSE_DEVICE *   pDevice   = NULL;
STATUS              status    = ERROR;
USBHST_STATUS       hstStatus = USBHST_SUCCESS;
size_t              nBytes    = 0;

if ((NULL == pUrb) || (NULL == pUrb->pContext))
    {
    USB2_MSE_ERR("Invalid Parameter\n", 1, 2, 3, 4, 5, 6);
    return USBHST_INVALID_PARAMETER;
    }

USB2_MSE_VDBG("usb2MseInUrbCompleteCallback(): "
              "URB returned with result %p\n",
              pUrb->nStatus, 2, 3, 4, 5, 6);

pDevice   = (USB2_MSE_DEVICE* ) pUrb->pContext;

pDevice->bTransferInCourse = FALSE;

status = usb2TakeDevice (pDevice->pClassDevice);

if (ERROR == status)
    {
    USB2_MSE_ERR("usb2MseInUrbCompleteCallback(): "
                 "Cannot take device ownership\n",1,2,3,4,5,6);

    return USBHST_FAILURE;
    }

 /*
 * Fill the buffer for what we have been getting, if the URB
 * has been transferred properly. If there is a transfer ERROR
 * with the URB, that means that THIS URB is not proper,
 * so ignore the content.We should only reporting
 * full packets, do not insert partial packets into the ring
 * buffer.
 */

if (USBHST_SUCCESS == pUrb->nStatus)
    {
    nBytes = rngBufPut(pDevice->hInRingBuffer,
                       (char*) pDevice->pReport,
                       sizeof(USB2_MSE_REPORT));

    if (pDevice->pReader)
        {
        if (0 < selWakeupListLen(pDevice->pSelWakeupList) )

            {
            selWakeupAll (pDevice->pSelWakeupList, SELREAD);
            }
        }

    /* Signal the waiting task that the transfer has finished */

    semGive (pDevice->hInSem);
```

```
    USB2_MSE_DBG("usb2MseInUrbCompleteCallback(): Inserted %d bytes"
                 "in the Device Ring buffer, %d bytes available \n",
                 nBytes, rngFreeBytes(pDevice->hInRingBuffer), 3, 4, 5, 6);
    }

/*
 * If there is no file descriptor,
 * suspend communicating with the device
 */

if (NULL == pDevice->pReader)
    {
    USB2_MSE_DBG("usb2MseInUrbCompleteCallback(): File descriptor has "
                 "been closed, return here without submitting \n",
                 1, 2, 3, 4, 5, 6);

    usb2ReleaseDevice(pDevice->pClassDevice);

    return USBHST_SUCCESS;
    }

USBHST_FILL_INTERRUPT_URB(pDevice->pInUrb,
                  pDevice->pClassDevice->hDevice,
                  pDevice->bInEndPointAddr,
                  (UCHAR*) pDevice->pReport,
                  pDevice->uReportSize,
                  USBHST_SHORT_TRANSFER_OK,
                  usb2MseInUrbCompleteCallback,
                  (void *) pDevice,
                  USBHST_SUCCESS
                  );

USB2_MSE_VDBG("usb2MseInUrbCompleteCallback(): "
              "Resubmitting URB...\n", 1, 2, 3, 4, 5, 6);

/* Prepare requesting data */

pDevice->bTransferInCourse = TRUE;

hstStatus = usbHstURBSubmit(pDevice->pInUrb);

if (USBHST_SUCCESS != hstStatus)
    {
    pDevice->bTransferInCourse = FALSE;

    /* Something wrong with the device */

    USB2_MSE_ERR("usb2MseInUrbCompleteCallback():"
                 "Submitting URB failed : %d\n", hstStatus, 2, 3, 4, 5, 6);
    }

USB2_MSE_VDBG("usb2MseInUrbCompleteCallback():  "
              "URB Submitted with Status %d\n", hstStatus, 2, 3, 4, 5, 6);

/* And return */
```

```
    usb2ReleaseDevice(pDevice->pClassDevice);

    return (hstStatus);
    }
```

7. Complete I/O System Open Routine

The I/O system open( ) routine for this USB mouse class driver example is called usb2MseIosOpen( ).

The complete usb2MseIosOpen( ) routine is shown below. For more information about this routine, see Implement the I/O System Open Routine on page 36.

```
LOCAL ULONG usb2MseIosOpen
    (
    DEV_HDR *   pDevHdr,
    char *      notUsed,
    int         flags,
    int         mode
    )
    {
    USB2_MSE_DEVICE *   pDevice = (USB2_MSE_DEVICE * ) pDevHdr;
    USB2_FILE_DESC *    pFd; /* File descriptor structure used */
    USBHST_STATUS       hstStatus;

    USB2_MSE_VDBG ("usb2MseIosOpen(): Opening the Mouse : "
                   "mode 0x%X, flags 0x%X\n", mode, flags, 3, 4, 5, 6);

    if (OK != usb2TakeDevice (pDevice->pClassDevice))
        {
        USB2_MSE_ERR ("usb2MseIosOpen(): "
                      "Cannot take device ownership\n",
                      1, 2, 3, 4, 5, 6);

        errnoSet(EINVAL);

        return (ULONG)ERROR;
        }

    /*
     * We don't want the device to be created and do not support
     * write to a mouse device, so refuse these flags
     */

    if ((O_RDWR & flags) || (O_WRONLY & flags) || (O_CREAT & flags ))
        {
        USB2_MSE_ERR("usb2MseIosOpen():"
                     "Device not writable/creatable \n",
                     1, 2, 3, 4, 5, 6);

        usb2ReleaseDevice(pDevice->pClassDevice);

        errnoSet(EINVAL);
```

```
        return (ULONG)ERROR;
        }

    /*
     * And we support only one fd reading at the time, so if the
     * pDevice->pReader is not NULL, then someone has already opened
     * the device, so we should return with ERROR here.
     */

    if (NULL != pDevice->pReader)
       {
       USB2_MSE_ERR("usb2MseIosOpen(): Driver only "
                    "allows one reader, another fd is open, return error \n",
                    1, 2, 3, 4, 5, 6);

       usb2ReleaseDevice (pDevice->pClassDevice);

       errnoSet (EINVAL);

       return (ULONG)ERROR;
       }

    /* Create the file descriptor structure */

    pFd = (USB2_FILE_DESC *) OSS_CALLOC (sizeof(USB2_FILE_DESC));
    if (NULL == pFd)
        {
        USB2_MSE_ERR ("usb2MseIosOpen():Cannot allocate data\n",
                      1, 2, 3, 4, 5, 6);

        usb2ReleaseDevice(pDevice->pClassDevice);

        errnoSet(ENOMEM);

        return (ULONG)ERROR;
        }

    /* Initialize the file descriptor structure */

    pFd->pDevice = pDevice;

    pFd->fdType = (UINT32)flags;

    pDevice->pReader = pFd;

    /* Flush the ring buffer so that no previous data is in it */

    rngFlush(pDevice->hInRingBuffer);

    USB2_MSE_DBG ("usb2MseIosOpen(): Device set to READ %p \n",
                      pFd, 2, 3, 4, 5, 6);

    USBHST_FILL_INTERRUPT_URB(pDevice->pInUrb,
                              pDevice->pClassDevice->hDevice,
                              pDevice->bInEndPointAddr,
                              (UCHAR*) pDevice->pReport,
                              pDevice->uReportSize,
```

```
                            USBHST_SHORT_TRANSFER_OK,
                            usb2MseInUrbCompleteCallback,
                            (void *) pDevice,
                            USBHST_SUCCESS
                            );

    USB2_MSE_VDBG("usb2MseIosOpen(): Submit an URB to start getting data \n",
                    1, 2, 3, 4, 5, 6);

    /*
     * Submitting the URB starts the I/O on this instance of the mouse
     */

    pDevice->bTransferInCourse = TRUE;

    if ((hstStatus = usbHstURBSubmit(pDevice->pInUrb)) != USBHST_SUCCESS)
        {
        USB2_MSE_ERR("usb2MseIosOpen():  usbHstURBSubmit failed %d\n",
                    hstStatus, 2, 3, 4, 5, 6);

        errnoSet (EIO);

        pDevice->bTransferInCourse = FALSE;

        pDevice->pReader = NULL;

        OSS_FREE (pFd);

        usb2ReleaseDevice(pDevice->pClassDevice);

        return (ULONG)ERROR;
        }

    usb2ReleaseDevice(pDevice->pClassDevice);

    return (ULONG) pFd;
    }
```